

---

# Singular Proxies for Adaptive Caching in Diffusion Language Models

---

Wenhao Sun<sup>1</sup> Rong-Cheng Tu<sup>§ 1</sup> Yifu Ding<sup>1</sup> Zhao Jin<sup>1</sup> Jingyi Liao<sup>1</sup> Yongcheng Jing<sup>1</sup> Dacheng Tao<sup>§ 1</sup>

## Abstract

While Diffusion Language Models (DLMs) offer a flexible, arbitrary-order alternative to the autoregressive paradigm, their non-causal nature precludes standard KV caching, forcing costly hidden state recomputation at every decoding step. Existing DLM caching approaches reduce this cost by selective hidden state updates; however, they are still limited by (i) costly token-wise update identification heuristics and (ii) rigid, uniform budget allocation that fails to account for heterogeneous hidden state dynamics. To address these challenges, we present SPA-Cache that jointly optimizes update identification and budget allocation in DLM cache. First, we derive a low-dimensional singular proxy that enables the identification of update-critical tokens in a low-dimensional subspace, substantially reducing the overhead of update identification. Second, we introduce an adaptive strategy that allocates fewer updates to stable layers without degrading generation quality. Together, these contributions significantly improve the efficiency of DLMs, yielding up to an  $8\times$  throughput improvement over vanilla decoding and a  $2\text{--}4\times$  speedup over existing caching baselines. The code is available at <https://github.com/wenhao728/spa-cache>.

## 1 Introduction

Diffusion Language Models (DLMs) (Austin et al., 2021a; Dieleman et al., 2022; Li et al., 2022; Gong et al., 2023) have recently emerged as a versatile alternative to the dominant autoregressive (AR) paradigm (Brown et al., 2020), primarily characterized by their ability to decode in arbitrary orders and facilitate the potential of parallel decoding.

---

<sup>1</sup>College of Computing and Data Science, Nanyang Technological University, Singapore, Singapore. Correspondence to: Rong-Cheng Tu <rongcheng.tu@ntu.edu.sg>, Dacheng Tao <dacheng.tao@ntu.edu.sg>.

By decoupling generation from a fixed left-to-right order, DLMs have demonstrated remarkable potential in handling multi-modality tasks (You et al., 2025; Yang et al., 2025), overcoming the “reverse curse,” (Berglund et al., 2024) enhancing diversity (Gong et al., 2025), and scaling effectively across reasoning (Zhao et al., 2025; Chen et al., 2025; Zhu et al., 2025; Liu et al., 2025a). However, a significant performance gap remains: while proprietary industrial prototype models, such as Gemini Diffusion (DeepMind, 2025) and Mercury (Khanna et al., 2025), have begun to showcase ultra high throughput, current open-source implementations (Nie et al., 2025; Ye et al., 2025) continue to suffer from prohibitive inference latency compared to their AR counterparts. This inefficiency is fundamentally rooted in the unpredictable decoding order and the resulting adaptation of bidirectional attention. Consequently, the standard KV-cache (Pope et al., 2023), a cornerstone of efficiency in AR models, is rendered incompatible, as the hidden states of DLMs cannot be simply reused without recomputation to capture the evolving global dependencies.

Recent efforts to address the DLM inference bottleneck have mainly focused on caching with heuristic-based updates, often assuming a locality bias, where only hidden states in the immediate vicinity of recently decoded tokens require updates (Ma et al., 2025; Jiang et al., 2025). Others rely on restrictive block-wise semi-autoregressive decoding (Wu et al., 2025b;a; Song et al., 2025), which compromises the arbitrary-order flexibility of DLMs and sacrifices global dependency variations. A more principled alternative (Liu et al., 2025b) attempts to identify update-critical tokens by monitoring shifts in the Value states; however, this approach remains bottlenecked. First, the identification process is computationally heavy, requiring high-dimensional projection and similarity computations at every step. This overhead often offsets the gains from sparse updates as the generation lengths scale. Second, these methods typically apply a uniform update budget across all layers. As illustrated in Figure 2, the fraction of drifting tokens, which necessitate updates, shows layer-wise heterogeneity. Consequently, a high fixed budget (e.g., 30%) leads to computational redundancy in stable layers, whereas a lower budget (e.g., 20%) triggers performance degradation due to the insufficient updating of highly unstable layers.

To address these challenges, we present **SPA-Cache**, a

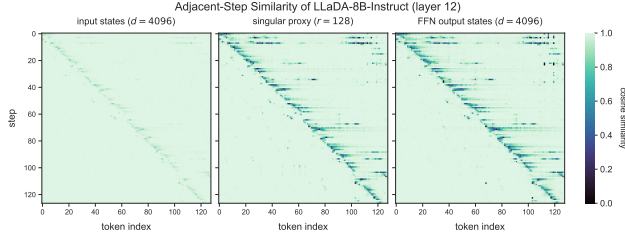


Figure 1. **Adjacent-Step Similarities for LLaDA-8B-Instruct** (Nie et al., 2025). While input states exhibit uniformly high similarity, our singular proxy at early stage of the layer can efficiently uncover the drifts in the final FFN output. This provides a clear guidance for selective hidden state updates.

framework that jointly optimizes update identification and budget allocation. We ground our approach in a formal analysis of the evolutionary dynamics of DLM hidden states, characterizing the statistical properties that dictate when and where updates are most critical. Building on these theoretical insights, we introduce a **singular proxy** that identifies update-critical tokens in the early stages of each layer (Figure 1). Leveraging a singular approximation (Stewart, 1993) of the identification process, our singular proxy retains the dominant directions of state drift in a low-dimensional manifold, thereby avoiding the computational bottleneck associated with high-dimensional identification. Furthermore, observing that hidden state stability fluctuates significantly across model layers, we depart from standard uniform update fractions in favor of an adaptive budget allocation scheme. By dynamically prioritizing computational resources for high-variance layers while aggressively caching stable ones, SPA-Cache maximizes throughput without sacrificing generation quality.

Extensive evaluations on seven benchmarks demonstrate that SPA-Cache significantly optimizes DLM inference, delivering up to a  $8\times$  increase in throughput with negligible impact on generation quality. Our SPA-Cache is fundamentally orthogonal to sequence-level optimizations and can be integrated with parallel decoding methods (Wu et al., 2025b) to amplify gains to a total of  $28\times$ . By drastically reducing inference overhead, this work paves the way and positions DLMs as a computationally viable alternative for high-throughput, real-world applications.

## 2 Related Work

### 2.1 Diffusion Language Models (DLMs)

The transition of diffusion processes to language modeling originated with Gaussian-based continuous latent diffusion in Diffusion-LM (Li et al., 2022) and structured discrete denoising in D3PM (Austin et al., 2021a). Subsequent frameworks such as CDCD (Dieleman et al., 2022) introduced score interpolation to bridge the discrete-continuous gap

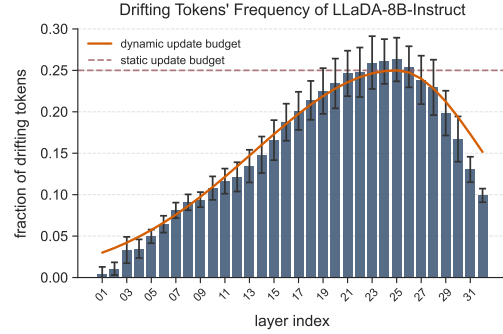


Figure 2. **Distribution of Drift Across Layers.** Rather than using a fixed threshold (dashed line) to allocate update budgets uniformly across layers, our SPA-Cache employs a dynamic threshold (solid line) that allocates more updates on the most volatile layers and fewer on the more stable layers.

between latents and targets. Plaid (Gulrajani & Hashimoto, 2023) demonstrated that diffusion models follow predictable scaling laws, eventually outperforming smaller autoregressive (AR) models in zero-shot likelihood. More recently, MDLM (Sahoo et al., 2024) and LLaDA (Nie et al., 2025) have converged on iterative unmasking as a highly scalable DLM alternative to AR modeling, offering bidirectional context and inherent parallelism. However, the bidirectional nature of these models requires recomputing a forward pass over the entire sequence of length  $N$  at each decoding step, resulting in a severe inference bottleneck with  $O(T \times N^2)$  complexity for  $T$  decoding steps.

### 2.2 Caching Mechanisms for Language Models

KV cache (Pope et al., 2023), a cornerstone of efficiency in AR language models, is fundamentally incompatible with DLMs due to the unpredictable decoding order and the subsequent reliance on full bidirectional attention (Liu et al., 2025b). Recent efforts, like dKV-Cache (Ma et al., 2025), d2Cache (Jiang et al., 2025), and Fast-dLLM (Wu et al., 2025b) propose window-based heuristics that selectively update tokens in the immediate vicinity of recently decoded ones while reusing distant hidden states. However, these methods often rely on a locality bias that lacks theoretical justification and may fail to capture complex global dependency variations. dLLM-Cache (Liu et al., 2025b) monitors feature drift to identify and update critical tokens at arbitrary positions. Despite its flexibility, this approach remains limited by (i) the high overhead of high-dimensional similarity checks and (ii) a uniform update budget across all layers that fails to exploit the layer-wise stability profiles of DLMs.

### 2.3 Other Efficient Generation Techniques

Beyond caching, researchers have explored cache eviction (Song et al., 2025) and specialized attention variants (Wang et al., 2025b; Zhang et al., 2025b) to reduce the cost of

**Algorithm 1** SPA-Cache of a Transformer Block

**input** Input states  $H$ , cached KV  $K^c$ ,  $V^c$ , cached FFN outputs  $H^c$ , layer index  $l$ , maximum update ratio  $\rho_p$

**output** Output states  $H$

**Phase 1: Update Identification & Selection**

$N \leftarrow \text{len}(H)$   $\triangleright$  Sequence length  $N$   
 $\rho \leftarrow \text{get\_ada\_rho}(\rho_p, l)$   $\triangleright$  Equation (5) (Section 3.4)  
 $\mathcal{I} \leftarrow \text{get\_update\_indices}(H, k = N \cdot \rho)$   $\triangleright$  Get update indices; Algorithm 2 (Sections 3.2 and 3.3)  
 $H_{\mathcal{I}} \leftarrow \text{gather}(H, \mathcal{I})$   $\triangleright \Pi_{\mathcal{I}}$ : Select states at indices  $\mathcal{I}$ ;  
 $\text{len}(H_{\mathcal{I}}) : k$

**Phase 2: Attention with Partially Cached KV**

$Q_{\mathcal{I}}, K_{\mathcal{I}}, V_{\mathcal{I}} \leftarrow \text{to\_qkv}(H_{\mathcal{I}})$   
 $K^c \leftarrow \text{scatter}(K^c, \mathcal{I}, K_{\mathcal{I}})$   
 $V^c \leftarrow \text{scatter}(V^c, \mathcal{I}, V_{\mathcal{I}})$   $\triangleright$  Upd: Update KV;  
 $\text{len}(K^c), \text{len}(V^c) : N$   
 $H_{\mathcal{I}} \leftarrow \text{Attn}(Q_{\mathcal{I}}, K^c, V^c)$

**Phase 3: FFN and Output States Update**

$H_{\mathcal{I}} \leftarrow \text{FFN}(H_{\mathcal{I}})$   
 $H^c \leftarrow H \leftarrow \text{scatter}(H^c, \mathcal{I}, H_{\mathcal{I}})$   $\triangleright$  Upd: Update output and its cache;  $\text{len}(H) : N$

bidirectional modeling. Furthermore, DLMs exhibit unique synergy with parallel decoding methods (Israel et al., 2025; Wang et al., 2025a; Wu et al., 2025b; Lanxiang et al., 2025), where confidence-aware strategies are adopted to simultaneously decode multiple tokens. Caching methods are fully compatible with these parallel frameworks. By integrating our adaptive caching, SPA-Cache, with these parallel frameworks, we can further amplify throughput gains.

### 3 Method

#### 3.1 Workflow Overview

As discussed in Section 2.1, the arbitrary-order decoding paradigm and the reliance on bi-directional self-attention in Diffusion Language Models (DLMs) render standard LM caching techniques inapplicable. To address this, we propose **SPA-Cache** for efficient DLM caching. The workflow for a single Transformer layer is partitioned into the following three distinct phases as shown in Algorithm 1 and Figure 3. Some operations (e.g., residual connections, position embeddings) are omitted for clarity.

**Phase 1: Update Identification and Input States Selection.** In this initial phase, the goal is to identify which tokens have drifted significantly to warrant recomputation. Since the drift in subsequent components is unknown beforehand, the challenge lies in determining the update priority efficiently. We employ an update identification Algorithm 2, which will be detailed in Sections 3.2 and 3.3, to select a set of update indices  $\mathcal{I}$ . These indices guide the selection of

**Algorithm 2** Update Identification

**input** Input states  $H = [h_1, \dots, h_N]$ , cached proxy identifiers  $P^c = [p_1^c, \dots, p_N^c]$ , update size  $k$

**output** Update indices  $\mathcal{I}$

**for**  $i = 1$  **to**  $N$  **do**

$p_i \leftarrow f_{\text{proxy}}(h_i)$   $\triangleright$  Compute new identifiers

$s_i \leftarrow \mathcal{S}_{\text{cos}}(p_i, p_i^c)$   $\triangleright$  Similarity scores

**end for**

$P \leftarrow [p_1, \dots, p_N]; S \leftarrow [s_1, \dots, s_N]$

$\mathcal{I} \leftarrow \text{topk}(S, k, \text{smallest}=\text{True})$   $\triangleright$  Indices to update

$P^c \leftarrow \text{scatter}(P^c, \mathcal{I}, P)$   $\triangleright$  Update identifiers

sparse inputs for computation while allowing the remaining tokens to stay dormant and use cache.

**Phase 2: Attention with Partially Cached Key-Value.**

The sparse inputs are used to generate a sparse Query  $Q_{\mathcal{I}}$ , along with Key  $K_{\mathcal{I}}$  and Value  $V_{\mathcal{I}}$  states exclusively for the tokens indexed in  $\mathcal{I}$ . The Update (Upd) module then integrates these new KV states into the existing KV cache from previous decoding steps. This process refreshes the representations of the active tokens while reusing cached states for stable tokens. The Attention is computed between the sparse queries and the partially updated KV cache, yielding the sparse attention output.

**Phase 3: FFN and Output States Update.** The final phase handles the feed-forward transitions. The sparse attention output is passed through a FFN component. Similar to Phase 2, an Update (Upd) module uses the indices  $\mathcal{I}$  to synchronize the newly computed output states with the cache and produce the final output states for the layer.

This selective recomputation mechanism ensures that the computational complexity of both the Attention and FFN components scales with the number of updated tokens  $k$  rather than the total sequence length  $N$ .

#### 3.2 On the Choice of Update Identifier

The core of our framework is the **update identification** mechanism in Phase 1, which selectively determines which tokens require recomputation. As detailed in Algorithm 2, we first project the input hidden states  $[h_i]$  into proxy identifier vectors  $[p_i]$  via a proxy projection  $f_{\text{proxy}}(\cdot)$ . To quantify temporal changes, we compute the cosine similarity  $\mathcal{S}_{\text{cos}}(\cdot)$  between the current identifier  $p_i$  and the cached identifier  $p_i^c$  from the previous decoding steps. The update set  $\mathcal{I}$  is then constructed by selecting the Top- $k$  tokens with the lowest similarity scores, effectively prioritizing those with the most significant state deviations.

Liu et al. (2025b) has empirically utilized Value states to guide the update of drifting states, i.e.  $p_i = f_{\text{proxy}}(h_i) = W h_i = v_i$ , where  $W \in \mathbb{R}^{d \times d}$  is the Value projection

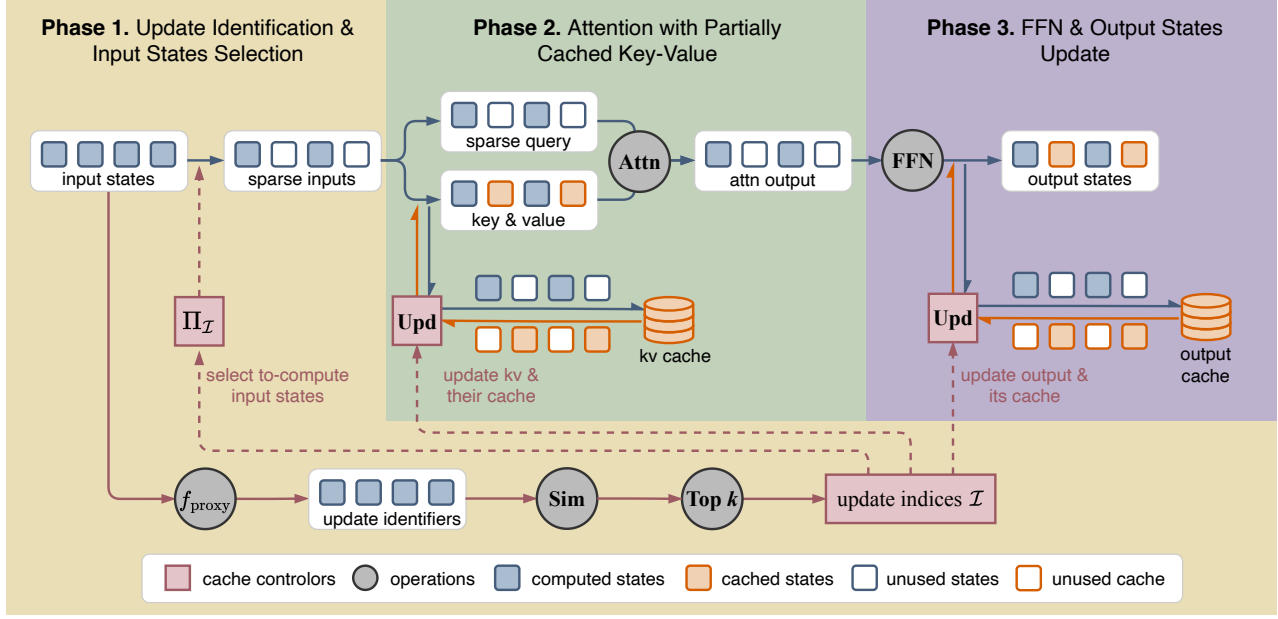


Figure 3. **Overview of a Single SPA-Cache Layer.** **Phase 1:** Input states are projected by  $f_{\text{proxy}}$  to produce update scores. A Top- $k$  selection identifies the update index set  $\mathcal{I}$ , which identifies positions whose cached states from the previous steps should be refreshed. **Phase 2:** Queries are generated only from the selected states and attend to a key-value cache that is selectively updated at indices  $\mathcal{I}$ . **Phase 3:** The resulting sparse attention outputs are fed into the MLP for the layer output. The output states is updated at  $\mathcal{I}$ , while non-selected positions reuse cached features.

matrix. However, such design choice was motivated by experimental correlations. In this section, we formalize the theoretical underpinnings of why Value states constitute a principled identifier for state updates. Specifically, we provide analyses of feature drift propagation through linear projections and non-linearities. Full proofs for the following theorems are provided in Appendix A.

We first establish that the stability of the Value states is a sufficient condition for the stability of the attention output.

**Theorem 3.1 (Bound on Attention Output Similarity).** Consider the Value states  $\mathbf{v}_i^t \in \mathbb{R}^d$  and attention output  $\mathbf{h}_i^t \in \mathbb{R}^d$  for the  $i$ -th token at decoding steps  $t$  of a DLM. Under the following assumptions:

1. **Bounded Norms:** The Value norms  $\|\mathbf{v}_i\|$  and attention output norms  $\|\mathbf{h}_i\|$  are bounded.
2. **Stable Attention:** The total variation of the attention weights is bounded:  $\sum_j |\alpha_{ij}^{t+1} - \alpha_{ij}^t| \leq \delta_A$ .
3. **Bounded Drift:** The drift of output states is controlled by its value state:  $\sum_j \alpha_{ij}^{t+1} \|\mathbf{v}_j^{t+1} - \mathbf{v}_j^t\| \leq \lambda \|\mathbf{v}_i^{t+1} - \mathbf{v}_i^t\|$ , where  $\lambda \geq 1$  is a relaxation constant.

Then, the cosine dissimilarity of the output  $\mathbf{h}_i$  is bounded by the dissimilarity of the  $i$ -th value state  $\mathbf{v}_i$ :

$$1 - \mathcal{S}_{\cos}(\mathbf{h}_i^t, \mathbf{h}_i^{t+1}) \leq C \cdot (1 - \mathcal{S}_{\cos}(\mathbf{v}_i^t, \mathbf{v}_i^{t+1})) + \epsilon, \quad (1)$$

where  $C > 0$  is a scaling constant determined by the norm bounds, and  $\epsilon > 0$  is a residual term capturing attention shifts and norm variance (detailed in Appendix A.1).

This implies that if the Value states maintain high similarity, the resulting attention output drift is upper bounded. Thus, monitoring Value states provides a conservative guarantee for the stability of the attention output.

Next, we argue that the FFN preserves this stability.

**Theorem 3.2 (Bound on FFN Output Divergence).** Let  $f_{\text{FFN}}(\cdot)$  be a standard FFN component comprising linear projections, RMSNorm (Zhang & Sennrich, 2019), and SiLU (Elfwing et al., 2018) activation functions. Let  $\mathbf{h}_1, \mathbf{h}_2 \in \mathbb{R}^d$  be two FFN input vectors. The divergence of the FFN outputs is bounded by the cosine similarity of their inputs:

$$\|f_{\text{FFN}}(\mathbf{h}_1) - f_{\text{FFN}}(\mathbf{h}_2)\|_2 \leq C \cdot \sqrt{1 - \mathcal{S}_{\cos}(\mathbf{h}_1, \mathbf{h}_2)} + \epsilon, \quad (2)$$

where  $C > 0$  and  $\epsilon > 0$  are constants depending on the Lipschitz constant of the network and the feature norms.

**Remark 3.3.** More generally, this theorem holds for normalizations that project onto a bounded hypersphere (e.g., LayerNorm (Ba et al., 2016)) and Lipschitz-continuous activations (e.g., GELU (Hendrycks & Gimpel, 2016)). The bounding property can also be extended to the more recent MoE (Zhang et al., 2025a) as they are intrinsically composed of standard FFNs.

Theorem 3.1 and Theorem 3.2 together provide the theoretical guarantee for the Value identification mechanism: if the Value states maintain high cosine similarity, the final layer output drifts are bounded. Consequently, it is unnecessary

Table 1. Comparison of different identifier types on **LLaDA-8B-Instruct** (Nie et al., 2025) and **GSM8K** (Cobbe et al., 2021). TPS: Tokens Per Second, TTFT: Time To First Token (in milliseconds).

| IDENTIFIER TYPE        | TPS $\uparrow$ | TTFT $\downarrow$ | ACCURACY $\uparrow$         |
|------------------------|----------------|-------------------|-----------------------------|
| <b>BASELINE (NONE)</b> | 29.01          | 28.6              | 78.62 ( $\pm 1.13$ )        |
| QUERY                  | 164.36         | 28.4              | 77.21 ( $\pm 1.15$ )        |
| KEY                    | 164.11         | 28.1              | 76.83 ( $\pm 1.15$ )        |
| VALUE                  | 164.88         | 28.1              | <b>78.59</b> ( $\pm 1.13$ ) |
| ATTN. INPUT            | <b>172.47</b>  | 28.6              | 77.29 ( $\pm 1.14$ )        |
| ATTN. OUTPUT           | 139.18         | 28.0              | 73.92 ( $\pm 1.21$ )        |

(and impractical) to speculatively evaluate the whole layer to detect divergence; the similarity of Value states act as a sufficient proxy for determining which tokens can safely reuse cached states.

**Empirical Evaluation of the Identifiers.** To validate our theoretical findings, we perform an empirical comparison of various hidden state components as update identifiers in Table 1. Specifically, we evaluate the efficacy when using Query, Key, Value, attention input, and attention output states as update identifiers. As demonstrated in Table 1, the Value state outperforms all other candidates, providing the most reliable signal for hidden state drift.

Query and Key serve as routing features; while shifts in their values may alter the attention score distribution, the resulting output remains stable if the attended Value vectors are semantically equivalent. The attention input state is an entangled representation of Query, Key, and Value states, making it less effective for isolating drift. Notably, using attention output states as identifiers yields the most significant accuracy degradation. This limitation stems from the Anisotropy Masking Effect (Ethayarajh, 2019; Gao et al., 2019; Timkey & van Schijndel, 2021), in which the representation space of deeper features collapses into a narrow cone. This dominance of a few directions masks the unique features necessary for robust identification. We provide an in-depth analysis of this failure mode in Appendix B.

### 3.3 Singular Proxy Identifier

Although the Value states are justified as a reliable identifier for update selection, projecting the whole input sequence to Values and computing similarity in the  $d$ -dimensional Value space are computationally expensive. In moderate-parameterized DLMs such as LLaDA-8B ( $d = 4096$ ), the identification process in this  $d$ -dimensional space incurs an overhead that can offset the gains from sparse computation.

To address this challenge, we introduce the **singular proxy**, a method that designed to identify update-critical tokens within a low-dimensional subspace  $r \ll d$ . We perform the Singular Value Decomposition (SVD) (Stewart, 1993) on the Value projection matrix  $W \in \mathbb{R}^{d \times d}$  such that

$W \approx U\Lambda V^\top$ , where  $U, V \in \mathbb{R}^{d \times d}$  denote the left and right singular vectors, and  $\Lambda = \text{diag}(\lambda_1, \dots, \lambda_d) \in \mathbb{R}^{d \times d}$  represents the singular values in descending order. We define a truncated projection  $W_r \in \mathbb{R}^{r \times d}$  by retaining the top  $r$  right singular vectors and singular values. This yields the dimension-reduction projection,  $f_{\text{proxy}}: \mathbb{R}^d \rightarrow \mathbb{R}^r$ :

$$f_{\text{proxy}}(\mathbf{h}_i) = W_r \mathbf{h}_i = (\Lambda_r V_r^\top) \mathbf{h}_i, \quad (3)$$

where  $\mathbf{h}_i \in \mathbb{R}^d$  is the  $i$ -th input state. By projecting the input into the principal subspace, we reduce the computational complexity of the identifier mapping  $f_{\text{proxy}}(\cdot)$  from  $O(d^3)$  to  $O(rd^2)$  and the similarity computation  $\mathcal{S}_{\text{cos}}(\cdot)$  from  $O(d)$  to  $O(r)$ . The remaining pipeline still follows the procedure outlined in Algorithm 2.

While truncated SVD preserves the majority of variance, it is critical to ensure that the topological structure remains intact within the reduced subspace. To formalize this, we establish the following bound on similarity divergence:

**Theorem 3.4 (Similarity Preservation in the Truncated Projection).** *Let  $\mathbf{h}_1, \mathbf{h}_2 \in \text{span}(V_r)$  be the input states. Denote  $\mathbf{v} = W\mathbf{h}$  and  $\hat{\mathbf{v}} = W_r\mathbf{h}$  as the Value states and our singular proxy, respectively. Their divergence of the cosine similarity is bounded by a factor independent of the input states  $\mathbf{h}_1, \mathbf{h}_2$ :*

$$|\mathcal{S}_{\text{cos}}(\mathbf{v}_1, \mathbf{v}_2) - \mathcal{S}_{\text{cos}}(\hat{\mathbf{v}}_1, \hat{\mathbf{v}}_2)| \leq 2 \left( \frac{\lambda_{r+1}}{\lambda_r} \right)^2. \quad (4)$$

**Remark 3.5.** The  $\mathbf{h}_1, \mathbf{h}_2 \in \text{span}(V_r)$  assumption typically holds as the cumulative update to each neuron can be viewed as a weighted summation of the training input vectors (Cheng et al., 2025). The rigorous proof is provided in Appendix A.3.

By leveraging Theorem 3.4, we can analytically ensure the singular proxy is an effective alternative for the Value proxy. This allows for computational reductions during the update identification process.

### 3.4 Adaptive Budget Allocation

While the singular proxy identifier in Section 3.3 reduces per-token identification overhead, the layer’s computation is also governed by the update ratio  $\rho$  (i.e., the fraction of tokens updated). Prior method (Liu et al., 2025b) typically employs a uniform budget allocation, maintaining a static update ratio (e.g.,  $\rho = 0.25$ ) across all layers. However, our analysis of hidden state evolution reveals significant layer-wise heterogeneity that renders uniform allocation less efficient.

As shown in Figure 2, we evaluate the average ratio of highly drifting tokens per layer (defined as tokens whose cosine similarity with last decoding step falls below a threshold  $\tau =$

Table 2. Comparison of LLaDA-8B-Instruct (Nie et al., 2025) and Dream-v0-Instruct-7B (Ye et al., 2025) across seven benchmarks. Compared to prior dLLM-Cache (Liu et al., 2025b) and Fast-dLLM (Wu et al., 2025b), our SPA-Cache achieves better speedups while maintaining comparable accuracy.

| TASK                  | METHOD       | LLADA-8B-INSTRUCT             |                        |                             | DREAM-V0-INSTRUCT-7B          |                        |                             |
|-----------------------|--------------|-------------------------------|------------------------|-----------------------------|-------------------------------|------------------------|-----------------------------|
|                       |              | TPS $\uparrow$                | TTFT (MS) $\downarrow$ | ACCURACY $\uparrow$         | TPS $\uparrow$                | TTFT (MS) $\downarrow$ | ACCURACY $\uparrow$         |
| MATHEMATICS & SCIENCE |              |                               |                        |                             |                               |                        |                             |
| GSM8K                 | BASLINE      | 29.67 <sub>(1.0\times)</sub>  | 28.0                   | 78.62 <sub>(\pm 1.13)</sub> | 17.86 <sub>(1.0\times)</sub>  | 23.6                   | 75.21 <sub>(\pm 1.19)</sub> |
|                       | + DLLM-CACHE | 68.62 <sub>(2.3\times)</sub>  | 28.7                   | 78.24 <sub>(\pm 1.14)</sub> | 23.39 <sub>(1.3\times)</sub>  | 25.3                   | 74.45 <sub>(\pm 1.20)</sub> |
|                       | + FAST-DLLM  | 93.86 <sub>(3.2\times)</sub>  | 28.1                   | 76.80 <sub>(\pm 1.16)</sub> | 25.53 <sub>(1.4\times)</sub>  | 24.6                   | 79.23 <sub>(\pm 1.12)</sub> |
|                       | + OURS       | 190.73 <sub>(6.4\times)</sub> | 28.7                   | 78.24 <sub>(\pm 1.14)</sub> | 45.97 <sub>(2.6\times)</sub>  | 24.5                   | 77.56 <sub>(\pm 1.15)</sub> |
| GPQA                  | BASLINE      | 20.55 <sub>(1.0\times)</sub>  | 44.8                   | 29.91 <sub>(\pm 2.17)</sub> | 12.89 <sub>(1.0\times)</sub>  | 61.2                   | 34.15 <sub>(\pm 2.24)</sub> |
|                       | + DLLM-CACHE | 27.29 <sub>(1.3\times)</sub>  | 44.9                   | 29.01 <sub>(\pm 2.15)</sub> | 25.40 <sub>(2.0\times)</sub>  | 61.4                   | 33.04 <sub>(\pm 2.22)</sub> |
|                       | + FAST-DLLM  | 25.91 <sub>(1.3\times)</sub>  | 44.4                   | 27.23 <sub>(\pm 2.11)</sub> | 30.63 <sub>(2.4\times)</sub>  | 61.3                   | 33.93 <sub>(\pm 2.24)</sub> |
|                       | + OURS       | 39.19 <sub>(1.9\times)</sub>  | 44.6                   | 30.58 <sub>(\pm 2.18)</sub> | 45.97 <sub>(3.6\times)</sub>  | 61.8                   | 34.60 <sub>(\pm 2.25)</sub> |
| MATH500               | BASLINE      | 33.35 <sub>(1.0\times)</sub>  | 23.3                   | 33.18 <sub>(\pm 0.62)</sub> | 23.08 <sub>(1.0\times)</sub>  | 20.2                   | 36.16 <sub>(\pm 0.63)</sub> |
|                       | + DLLM-CACHE | 74.26 <sub>(2.2\times)</sub>  | 23.2                   | 33.14 <sub>(\pm 0.62)</sub> | 42.08 <sub>(1.8\times)</sub>  | 21.9                   | 32.86 <sub>(\pm 0.63)</sub> |
|                       | + FAST-DLLM  | 85.94 <sub>(2.6\times)</sub>  | 23.2                   | 32.02 <sub>(\pm 0.62)</sub> | 89.05 <sub>(3.9\times)</sub>  | 20.9                   | 35.04 <sub>(\pm 0.63)</sub> |
|                       | + OURS       | 172.19 <sub>(5.2\times)</sub> | 24.8                   | 33.44 <sub>(\pm 0.62)</sub> | 104.23 <sub>(4.5\times)</sub> | 20.8                   | 34.84 <sub>(\pm 0.63)</sub> |
| GENERAL QA            |              |                               |                        |                             |                               |                        |                             |
| BBH                   | BASLINE      | 24.85 <sub>(1.0\times)</sub>  | 16.2                   | 53.49 <sub>(\pm 0.54)</sub> | 49.30 <sub>(1.0\times)</sub>  | 14.0                   | 58.42 <sub>(\pm 0.52)</sub> |
|                       | + DLLM-CACHE | 40.48 <sub>(1.6\times)</sub>  | 16.5                   | 52.74 <sub>(\pm 0.55)</sub> | 54.65 <sub>(1.1\times)</sub>  | 14.0                   | 58.22 <sub>(\pm 0.52)</sub> |
|                       | + FAST-DLLM  | 105.82 <sub>(4.3\times)</sub> | 17.0                   | 54.66 <sub>(\pm 0.56)</sub> | 123.05 <sub>(2.5\times)</sub> | 13.2                   | 57.79 <sub>(\pm 0.52)</sub> |
|                       | + OURS       | 108.31 <sub>(4.4\times)</sub> | 16.1                   | 53.72 <sub>(\pm 0.55)</sub> | 153.61 <sub>(3.1\times)</sub> | 13.2                   | 58.52 <sub>(\pm 0.52)</sub> |
| MMLU-PRO              | BASLINE      | 20.68 <sub>(1.0\times)</sub>  | 43.8                   | 37.08 <sub>(\pm 0.43)</sub> | 10.06 <sub>(1.0\times)</sub>  | 33.9                   | 45.01 <sub>(\pm 0.44)</sub> |
|                       | + DLLM-CACHE | 52.71 <sub>(2.5\times)</sub>  | 43.5                   | 36.50 <sub>(\pm 0.43)</sub> | 15.53 <sub>(1.5\times)</sub>  | 34.3                   | 44.16 <sub>(\pm 0.44)</sub> |
|                       | + FAST-DLLM  | 81.25 <sub>(3.9\times)</sub>  | 44.1                   | 37.28 <sub>(\pm 0.43)</sub> | 50.58 <sub>(5.0\times)</sub>  | 34.2                   | 46.62 <sub>(\pm 0.44)</sub> |
|                       | + OURS       | 124.06 <sub>(6.0\times)</sub> | 44.2                   | 36.30 <sub>(\pm 0.43)</sub> | 62.13 <sub>(6.2\times)</sub>  | 35.0                   | 45.16 <sub>(\pm 0.44)</sub> |
| CODE                  |              |                               |                        |                             |                               |                        |                             |
| MBPP                  | BASLINE      | 5.75 <sub>(1.0\times)</sub>   | 28.7                   | 39.20                       | 36.51 <sub>(1.0\times)</sub>  | 22.2                   | 57.40                       |
|                       | + DLLM-CACHE | 8.38 <sub>(1.5\times)</sub>   | 28.8                   | 39.00                       | 50.31 <sub>(1.4\times)</sub>  | 21.9                   | 56.20                       |
|                       | + FAST-DLLM  | 12.49 <sub>(2.2\times)</sub>  | 28.6                   | 37.80                       | 58.42 <sub>(1.6\times)</sub>  | 21.7                   | 51.00                       |
|                       | + OURS       | 46.12 <sub>(8.0\times)</sub>  | 28.7                   | 39.00                       | 114.85 <sub>(3.1\times)</sub> | 22.6                   | 57.60                       |
| HUMANEval             | BASLINE      | 37.48 <sub>(1.0\times)</sub>  | 14.4                   | 42.07                       | 28.00 <sub>(1.0\times)</sub>  | 14.0                   | 58.54                       |
|                       | + DLLM-CACHE | 40.29 <sub>(1.1\times)</sub>  | 14.3                   | 39.63                       | 21.15 <sub>(0.8\times)</sub>  | 14.1                   | 45.12                       |
|                       | + FAST-DLLM  | 81.90 <sub>(2.2\times)</sub>  | 14.3                   | 41.71                       | 40.34 <sub>(1.4\times)</sub>  | 13.8                   | 53.05                       |
|                       | + OURS       | 132.91 <sub>(3.5\times)</sub> | 14.3                   | 42.07                       | 42.49 <sub>(1.5\times)</sub>  | 13.4                   | 58.54                       |

0.95). This analysis, conducted over 100 random samples from GSM8K (Cobbe et al., 2021), MMLU-pro (Wang et al., 2024), and MBPP (Austin et al., 2021b), demonstrates a consistent distribution across diverse tasks. Specifically, we observe that state drift varies drastically across the model depth: (1) Hidden states remain relatively stable as they primarily perform initial embedding transformations in the early layers; (2) Drift peaks as the model performs complex semantic transformations in the middle layers, requiring more frequent updates; (3) Dynamics stabilize again as approaching the final layer of the model. A fixed update threshold (e.g.,  $\rho = 0.25$ , represented by the dashed line in Figure 2) essentially over-allocates budget to the stable early layers and late layers.

To exploit this heterogeneity, SPA-Cache replaces rigid allocation with an adaptive budget allocation scheme. Noticing that the drift curve roughly follows an asymmetric bell shape, we parameterize the layer-wise dynamic update ratio

$\rho(l)$  using a piecewise Gaussian function:

$$\rho(l) = \begin{cases} \rho_p \cdot \exp\left(\ln\left(\frac{\rho_1}{\rho_p}\right) \cdot \left(\frac{l-l_p}{l_p-1}\right)^2\right) & \text{if } l \leq l_p \\ \rho_p \cdot \exp\left(\ln\left(\frac{\rho_L}{\rho_p}\right) \cdot \left(\frac{l-l_p}{L-l_p}\right)^2\right) & \text{if } l > l_p \end{cases}, \quad (5)$$

where,  $l$  denotes the current layer index.  $L$  is the total layer number of the model.  $\rho_p$  is the peak update ratio at layer  $l_p$ . And  $\rho_1, \rho_L$  represent the boundary update ratios for the first and last layers, respectively. This parameterization provides the flexibility to adapt to various DLMs with different skewness in their profiles. More analysis and configurations are detailed in Appendix C.

The dynamic strategy ensures that computational resources are concentrated on high-variance layers where updates are most critical for preserving accuracy, while aggressively caching stable layers to maximize throughput.

## 4 Experiments

### 4.1 Experimental Setup

**Models and Benchmarks.** We evaluate SPA-Cache across two representative Diffusion Language Models (DLMs): LLaDA-8B (Nie et al., 2025) and Dream-7B (Ye et al., 2025). To assess generalizability, we conduct evaluations on seven diverse benchmarks covering mathematical reasoning (GSM8K (Cobbe et al., 2021), MATH500 (Hendrycks et al., 2021)), scientific knowledge (GPQA (Rein et al., 2023)), general-purpose QA (BBH (Suzgun et al., 2023), MMLU-pro (Wang et al., 2024)), and code generation (MBPP (Austin et al., 2021b), HumanEval (Chen et al., 2021)).

**Baselines.** We compare SPA-Cache against vanilla DLM decoding (no cache) and two state-of-the-art caching frameworks: dLLM-Cache (Liu et al., 2025b) and Fast-dLLM (Wu et al., 2025b).

**Implementation Details.** All experiments are executed on a single NVIDIA B200 GPU. For all experiments, we adopt the configuration from and set the allocation hyperparameter to  $\rho_p = 0.25$ . We report accuracy (for QA/Reasoning) and pass@1 (for coding tasks). To quantify efficiency gains, we measure average Tokens Per Second (TPS) and Time-To-First-Token (TTFT). Further implementation details and additional results on the post-trained LLaDA-1.5 (Zhu et al., 2025) are provided in Appendix D and E, respectively.

### 4.2 Main Results

Table 2 shows that SPA-Cache consistently outperforms existing caching methods across all benchmarks. By leveraging both the singular proxy identifier and adaptive budget allocation SPA-Cache achieves substantial throughput improvements and maintains comparable accuracy to vanilla decoding. Specifically, on the MMLU-pro benchmark, SPA-Cache yields a  $6.0\times$  and  $6.2\times$  speedup on LLaDA-8B and Dream-7B, respectively. The most significant gain is observed on MBPP with LLaDA-8B, where SPA-Cache attains an  $8.0\times$  throughput acceleration.

### 4.3 Integration with Parallel Decoding.

An advantage of DLMs is their inherent potentiality for parallel decoding (Nie et al., 2025). SPA-Cache is orthogonal to these techniques. To demonstrate this synergy, we integrate SPA-Cache with the parallel decoding framework from Wu et al. (2025b).

As shown in Table 3, combining SPA-Cache with parallelization yields further efficiency gains. While parallel decoding introduces a marginal distribution approximation error (resulting in a slight accuracy trade-off), the throughput improvements are remarkable. On the BBH benchmark, the combined approach achieves an unprecedented  $28\times$

Table 3. Integration of SPA-Cache with parallel strategy (Wu et al., 2025b). Our approach outperforms the Fast-dLLM dual cache in throughput while preserving comparable accuracy.

| TASK                  | METHOD      | TPS $\uparrow$          | ACCURACY $\uparrow$  |
|-----------------------|-------------|-------------------------|----------------------|
| MATHEMATICS & SCIENCE |             |                         |                      |
| GSM8K                 | BASILINE    | 29.67 (1.0 $\times$ )   | 78.62 ( $\pm 1.13$ ) |
|                       | + FAST-DLLM | 176.45 (5.9 $\times$ )  | 77.02 ( $\pm 1.16$ ) |
|                       | + OURS      | 276.39 (9.3 $\times$ )  | 76.81 ( $\pm 1.18$ ) |
| GPQA                  | BASILINE    | 20.55 (1.0 $\times$ )   | 29.91 ( $\pm 2.17$ ) |
|                       | + FAST-DLLM | 44.40 (2.2 $\times$ )   | 27.68 ( $\pm 2.12$ ) |
|                       | + OURS      | 47.82 (2.3 $\times$ )   | 27.01 ( $\pm 2.10$ ) |
| MATH500               | BASILINE    | 33.35 (1.0 $\times$ )   | 33.18 ( $\pm 0.62$ ) |
|                       | + FAST-DLLM | 86.69 (2.6 $\times$ )   | 31.76 ( $\pm 0.62$ ) |
|                       | + OURS      | 289.71 (8.7 $\times$ )  | 32.66 ( $\pm 0.63$ ) |
| GENERAL QA            |             |                         |                      |
| BBH                   | BASILINE    | 24.85 (1.0 $\times$ )   | 51.49 ( $\pm 0.54$ ) |
|                       | + FAST-DLLM | 301.33 (12.1 $\times$ ) | 53.20 ( $\pm 0.56$ ) |
|                       | + OURS      | 693.96 (27.9 $\times$ ) | 53.13 ( $\pm 0.56$ ) |
| MMLU-PRO              | BASILINE    | 20.68 (1.0 $\times$ )   | 37.08 ( $\pm 0.43$ ) |
|                       | + FAST-DLLM | 86.40 (4.2 $\times$ )   | 37.34 ( $\pm 0.43$ ) |
|                       | + OURS      | 224.97 (10.9 $\times$ ) | 36.85 ( $\pm 0.42$ ) |
| CODE                  |             |                         |                      |
| MBPP                  | BASILINE    | 5.75 (1.0 $\times$ )    | 39.20                |
|                       | + FAST-DLLM | 50.11 (8.7 $\times$ )   | 37.80                |
|                       | + OURS      | 143.25 (24.9 $\times$ ) | 38.20                |
| HUMAN-EVAL            | BASILINE    | 37.48 (1.0 $\times$ )   | 42.07                |
|                       | + FAST-DLLM | 117.67 (3.1 $\times$ )  | 39.81                |
|                       | + OURS      | 215.83 (5.8 $\times$ )  | 40.24                |

Table 4. Ablation on identifier and adaptive budget allocation.

| IDENTIFIER              | PEAK $\rho_p$ | AVG $\bar{\rho}$ | TPS $\uparrow$ | ACCURACY $\uparrow$         |
|-------------------------|---------------|------------------|----------------|-----------------------------|
| NONE                    | 100%          | 100%             | 29.01          | 78.62 ( $\pm 1.13$ )        |
| VALUE                   | 25%           | 25%              | 164.88         | <b>78.59</b> ( $\pm 1.13$ ) |
| SINGULAR <sub>128</sub> | 25%           | 25%              | 179.43         | 78.23 ( $\pm 1.12$ )        |
| SINGULAR <sub>128</sub> | 25%           | 16%              | <b>189.13</b>  | 78.24 ( $\pm 1.14$ )        |
| SINGULAR <sub>128</sub> | 16%           | 16%              | <b>190.06</b>  | 75.65 ( $\pm 1.17$ )        |

speedup over vanilla decoding. Crucially, SPA-Cache continues to outperform Fast-dLLM’s dual cache within the parallelized setting, showcasing its robustness.

### 4.4 Ablations

In this section, we conduct extensive ablation studies to systematically evaluate the individual contributions of our proposed components to the overall performance of SPA-Cache in practice. Unless otherwise specified, all experiments in this section are performed using the LLaDA-8B model (Nie et al., 2025) on the GSM8K (Cobbe et al., 2021) benchmark.

**Efficacy of Proposed Components.** We isolate the contributions of the singular proxy and adaptive allocation in Table 4. Replacing the Value proxy identifier with our singular proxy identifier (Section 3.3) increases TPS from 165 to 179 without degrading accuracy. Furthermore, introducing the adaptive budget allocation (Section 3.4) allows the

Table 5. **Impact of Proxy Rank  $r$  on LLaDA-8B-Instruct (Zhu et al., 2025), GSM8K (Cobbe et al., 2021).** While the full-dimensional Value identifier uses  $d = 4096$ , our singular value proxy achieves comparable performance at  $r = 512$ . Notably,  $r = 128$  strike a good balance between throughput and accuracy.

| IDENTIFIER              | TPS $\uparrow$ | ACCURACY $\uparrow$  |
|-------------------------|----------------|----------------------|
| NONE                    | 29.01          | 78.62 ( $\pm 1.13$ ) |
| VALUE                   | 164.88         | 78.59 ( $\pm 1.13$ ) |
| SINGULAR <sub>512</sub> | 172.56         | 78.57 ( $\pm 1.13$ ) |
| SINGULAR <sub>256</sub> | 176.38         | 78.47 ( $\pm 1.12$ ) |
| SINGULAR <sub>128</sub> | 179.43         | 78.23 ( $\pm 1.12$ ) |
| SINGULAR <sub>64</sub>  | 181.83         | 77.79 ( $\pm 1.14$ ) |
| SINGULAR <sub>32</sub>  | 182.66         | 76.71 ( $\pm 1.15$ ) |

model to reduce the average update ratio from  $\bar{\rho} = 25\%$  to  $\bar{\rho} = 16\%$ , pushing the TPS to 189. Notably, when we ignore the layer-wise heterogeneity and force a uniform 16% update ratio across all layers, accuracy drops from 78.23% to 75.56%. This confirms that our adaptive strategy successfully identifies and exploits stable layers to save computation without sacrificing model performance.

**Impact of Singular Proxy Rank.** The dimension of the singular proxy ( $r$ ) controls the trade-off between identification overhead and approximation quality. We quantitatively analyze this in Table 5. We observe that while lower dimensions further improve TPS, accuracy begins to decline when  $r < 128$ . We therefore select  $r = 128$  as the sweet spot and use it as the default value across all experiments.

**Analysis of Computational Overhead.** To investigate the source of our efficiency gains, we profile the per-layer breakdown for LLaDA-8B in Figure 4 (setting prompt length 1024, decoding length 256, and  $\rho = 5\%$ ). In the vanilla baseline, execution time is dominated by the linear projections and non-linearities in the Attention and FFN components. While the Value proxy caching methods successfully bypass these primary costs, they introduce a non-trivial identification overhead that emerges as a new computational bottleneck. In contrast, SPA-Cache’s singular proxy enables token selection in a highly compressed subspace, drastically reducing identification latency. This optimization allows the total execution time to approach the theoretical floor, maxi-

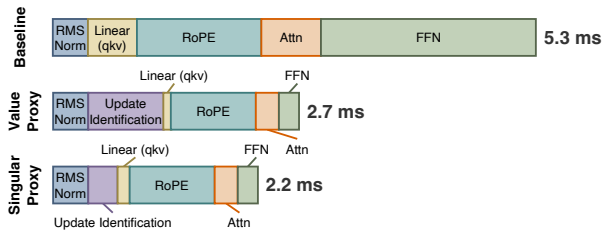


Figure 4. **Component-wise Latency Decomposition.** Value proxy reduces Attention and FFN overhead, but it introduces additional computational cost during update identification. Our proposed singular proxy further reduces this identification cost.

mizing the practical utility of the sparse update strategy.

## 5 Discussion

**Alternative Design Explorations.** We investigated another alternatives that ultimately proved sub-optimal. In current DLM implementations with moderate sequence lengths (*e.g.*  $\leq 2048$ ), the Attention is relatively lightweight compared to the FFN component. We hypothesized that using the Attention output as an identifier would provide a higher-fidelity signal for update selection. However, as shown in Table 1, identification accuracy dropped significantly. Post-mortem analysis suggests this is due to the Anisotropy Masking Effect (see Section B), where the Attention output exhibits a lower signal-to-noise ratio for hidden state dynamics.

**Limitations.** While SPA-Cache demonstrates superior performance across various benchmarks, it possesses certain limitations. Like other DLM caching methods (Liu et al., 2025b; Wu et al., 2025b; Ma et al., 2025), its efficacy relies on the temporal smoothness of hidden state transitions. In high-temperature sampling regimes (*e.g.*  $\tau \in [1.0, 1.5]$ ), the increased stochasticity can lead to reduced accuracy. Additionally, deploying SPA-Cache in distributed or tensor-parallel settings may require advanced engineering specifications, increasing the implementation complexity.

**Further study.** SPA-Cache primarily optimizes the decoding phase, leaving the prefilling latency (Time-to-First-Token, TTFT) largely unaffected. Furthermore, as current open-source DLMs do not yet support extended context windows, evaluating their performance on ultra-long sequences (*e.g.*,  $> 32k$  tokens) will be a critical area of investigation as foundation models evolve to incorporate such capabilities.

## 6 Conclusion

This paper introduces SPA-Cache, a training-free and principled caching framework designed to address the inference bottlenecks of diffusion language models. Moving beyond purely empirical observations, we provide a theoretical foundation for hidden state update identification, justifying the use of Value states as an effective proxy. By projecting the identification process into a low-dimensional manifold through our singular proxy approach, we significantly reduce overhead of update identification. Furthermore, we capitalize on the discovered layer-wise heterogeneity, utilizing an adaptive budget allocation strategy to avoid redundant computations in stable layers. Extensive benchmarks confirm that SPA-Cache delivers substantial throughput gains without compromising generation fidelity, and its seamless integration with parallel decoding underscores its versatility. We believe SPA-Cache establishes a new efficiency baseline and will catalyze further research into the practical deployment of discrete diffusion models.

## Acknowledgements

This research / project is supported by the National Research Foundation, Singapore, and Cyber Security Agency of Singapore under its National Cybersecurity R&D Programme and CyberSG R&D Cyber Research Programme Office. Any opinions, findings and conclusions or recommendations expressed in these materials are those of the author(s) and do not reflect the views of National Research Foundation, Singapore, Cyber Security Agency of Singapore as well as CyberSG R&D Programme Office, Singapore.

## Impact Statement

Generative language models carry the risk of producing biased, privacy-violating, harmful content, or offending intellectual property. Our method, designed to improve the generation efficiency of diffusion language models, may also inherit these potential negative impacts. Users and service providers should take responsibility for the generated content and strive to ensure positive social impacts.

## References

- Ainslie, J., Lee-Thorp, J., de Jong, M., Zemlyanskiy, Y., Lebrón, F., and Sanghai, S. GQA: training generalized multi-query transformer models from multi-head checkpoints. In *EMNLP*, pp. 4895–4901. Association for Computational Linguistics, 2023.
- Austin, J., Johnson, D. D., Ho, J., Tarlow, D., and van den Berg, R. Structured denoising diffusion models in discrete state-spaces. In *NeurIPS*, pp. 17981–17993, 2021a.
- Austin, J., Odena, A., Nye, M. I., Bosma, M., Michalewski, H., Dohan, D., Jiang, E., Cai, C. J., Terry, M., Le, Q. V., and Sutton, C. Program synthesis with large language models. *CoRR*, abs/2108.07732, 2021b.
- Ba, L. J., Kiros, J. R., and Hinton, G. E. Layer normalization. *CoRR*, abs/1607.06450, 2016.
- Berglund, L., Tong, M., Kaufmann, M., Balesni, M., Stickland, A. C., Korbak, T., and Evans, O. The reversal curse: LMs trained on "a is b" fail to learn "b is a". In *ICLR*, 2024.
- Brown, T. B., Mann, B., Ryder, N., Subbiah, M., Kaplan, J., Dhariwal, P., Neelakantan, A., and Others. Language models are few-shot learners. In *NeurIPS*, 2020.
- Chen, M., Tworek, J., Jun, H., Yuan, Q., de Oliveira Pinto, H. P., and Others. Evaluating large language models trained on code. *CoRR*, abs/2107.03374, 2021.
- Chen, R., Chai, W., Yang, Z., Zhang, X., Wang, Z., Quek, T. Q. S., Zhou, J. T., Poria, S., and Liu, Z. Diffpo: Diffusion-styled preference optimization for inference time alignment of large language models. In *ACL*, pp. 18910–18925. Association for Computational Linguistics, 2025.
- Cheng, R., Xiong, F., Wei, Y., Zhu, W., and Yuan, C. Whoever started the interference should end it: Guiding data-free model merging via task vectors. In *ICML*, 2025.
- Cobbe, K., Kosaraju, V., Bavarian, M., Chen, M., Jun, H., Kaiser, L., Plappert, M., Tworek, J., Hilton, J., Nakano, R., Hesse, C., and Schulman, J. Training verifiers to solve math word problems. *CoRR*, abs/2110.14168, 2021.
- DeepMind, G. Gemini diffusion: Our state-of-the-art, experimental text diffusion model. <https://deepmind.google/models/gemini-diffusion>, 2025. Accessed: 2025-12-30.
- Dieleman, S., Sartran, L., Roshannai, A., Savinov, N., Ganin, Y., Richemond, P. H., Doucet, A., Strudel, R., Dyer, C., Durkan, C., Hawthorne, C., Leblond, R., Grathwohl, W., and Adler, J. Continuous diffusion for categorical data. *CoRR*, abs/2211.15089, 2022.
- Elfwing, S., Uchibe, E., and Doya, K. Sigmoid-weighted linear units for neural network function approximation in reinforcement learning. *Neural Networks*, 107:3–11, 2018.
- Ethayarajh, K. How contextual are contextualized word representations? comparing the geometry of bert, elmo, and GPT-2 embeddings. In *EMNLP/IJCNLP*, pp. 55–65. Association for Computational Linguistics, 2019.
- Fang, W., Liu, S., Zhou, Y., Zhang, K., Zheng, T., Chen, K., Song, M., and Tao, D. Serl: Self-play reinforcement learning for large language models with limited data. *CoRR*, abs/2505.20347, 2025.
- Gao, J., He, D., Tan, X., Qin, T., Wang, L., and Liu, T. Representation degeneration problem in training natural language generation models. In *ICLR*, 2019.
- Gong, S., Li, M., Feng, J., Wu, Z., and Kong, L. Diffuseq: Sequence to sequence text generation with diffusion models. In *ICLR*, 2023.
- Gong, S., Zhang, R., Zheng, H., Gu, J., Jaitly, N., Kong, L., and Zhang, Y. Diffucoder: Understanding and improving masked diffusion models for code generation. *CoRR*, abs/2506.20639, 2025.
- Gulrajani, I. and Hashimoto, T. B. Likelihood-based diffusion language models. In *NeurIPS*, 2023.
- Hendrycks, D. and Gimpel, K. Bridging nonlinearities and stochastic regularizers with gaussian error linear units. *CoRR*, abs/1606.08415, 2016.

- Hendrycks, D., Burns, C., Kadavath, S., Arora, A., Basart, S., Tang, E., Song, D., and Steinhardt, J. Measuring mathematical problem solving with the MATH dataset. In *NeurIPS Datasets and Benchmarks*, 2021.
- Henry, G., Eibe, F., Bernhard, P., and Michael, C. Regularisation of neural networks by enforcing lipschitz continuity. *CoRR*, abs/1804.04368, 2018.
- Israel, D., den Broeck, G. V., and Grover, A. Accelerating diffusion llms via adaptive parallel decoding. *CoRR*, abs/2506.00413, 2025.
- Jacob, T. and Ambuj, T. Sequence length independent norm-based generalization bounds for transformers. In *AISTATS*, pp. 1405–1413. PMLR, 2024.
- Jiang, Y., Cai, Y., Luo, X., Fu, J., Wang, J., Liu, C., and Yang, X. d<sup>2</sup>cache: Accelerating diffusion-based llms via dual adaptive caching. *CoRR*, abs/2509.23094, 2025.
- Khanna, S., Kharbanda, S., Li, S., Varma, H., Wang, E., Birnbaum, S., Luo, Z., Miraoui, Y., Palrecha, A., Ermon, S., Grover, A., and Kuleshov, V. Mercury: Ultra-fast language models based on diffusion. *CoRR*, abs/2506.17298, 2025.
- Lanxiang, H., Siqu, K., Yichao, F., Samyam, R., Tajana, R., Yuxiong, H., Zhijie, D., and Hao, Z. Fast and accurate causal parallel decoding using jacobi forcing. *CoRR*, abs/2512.14681, 2025.
- Li, X. L., Thickstun, J., Gulrajani, I., Liang, P., and Hashimoto, T. B. Diffusion-lm improves controllable text generation. In *NeurIPS*, 2022.
- Liu, S., Fang, W., Hu, Z., Zhang, J., Zhou, Y., Zhang, K., Tu, R., Lin, T., Huang, F., Song, M., Li, Y., and Tao, D. A survey of direct preference optimization. *CoRR*, abs/2503.11701, 2025a.
- Liu, Z., Yang, Y., Zhang, Y., Chen, J., Zou, C., Wei, Q., Wang, S., and Zhang, L. dllm-cache: Accelerating diffusion large language models with adaptive caching. *CoRR*, abs/2506.06295, 2025b.
- Ma, X., Yu, R., Fang, G., and Wang, X. dkv-cache: The cache for diffusion language models. *CoRR*, abs/2505.15781, 2025.
- Nguyen-Tri, Q., Ranjan, M., and Shen, Z. Attention is all you need for KV cache in diffusion llms. *CoRR*, abs/2510.14973, 2025.
- Nie, S., Zhu, F., You, Z., Zhang, X., Ou, J., Hu, J., Zhou, J., Lin, Y., Wen, J., and Li, C. Large language diffusion models. *CoRR*, abs/2502.09992, 2025.
- Pope, R., Douglas, S., Chowdhery, A., Devlin, J., Bradbury, J., Heek, J., Xiao, K., Agrawal, S., and Dean, J. Efficiently scaling transformer inference. In *MLSys*. mlsys.org, 2023.
- Rein, D., Hou, B. L., Stickland, A. C., Petty, J., Pang, R. Y., Dirani, J., Michael, J., and Bowman, S. R. GPQA: A graduate-level google-proof q&a benchmark. *CoRR*, abs/2311.12022, 2023.
- Ruibin, X., Yunchang, Y., Di, H., Kai, Z., Shuxin, Z., Chen, X., Huishuai, Z., Yanyan, L., Liwei, W., and Tieyan, L. On layer normalization in the transformer architecture. In *ICML*, pp. 10524–10533. PMLR, 2020.
- Sahoo, S. S., Arriola, M., Schiff, Y., Gokaslan, A., Marroquin, E., Chiu, J. T., Rush, A., and Kuleshov, V. Simple and effective masked diffusion language models. In *NeurIPS*, 2024.
- Song, Y., Liu, X., Li, R., Liu, Z., Huang, Z., Guo, Q., He, Z., and Qiu, X. Sparse-dllm: Accelerating diffusion llms with dynamic cache eviction. *CoRR*, abs/2508.02558, 2025.
- Steven, B., Mazeika, M., Mostajabi, M., Steinhardt, J., and Song, D. Scaling out-of-distribution detection for real-world settings. In *ICML*, 2022.
- Stewart, G. W. On the early history of the singular value decomposition. *SIAM Rev.*, 35(4):551–566, 1993.
- Suzgun, M., Scales, N., Schärli, N., Gehrmann, S., Tay, Y., Chung, H. W., Chowdhery, A., Le, Q. V., Chi, E. H., Zhou, D., and Wei, J. Challenging big-bench tasks and whether chain-of-thought can solve them. In *ACL (Findings)*, pp. 13003–13051. Association for Computational Linguistics, 2023.
- Timkey, W. and van Schijndel, M. All bark and no bite: Rogue dimensions in transformer language models obscure representational quality. In *EMNLP*, pp. 4527–4546. Association for Computational Linguistics, 2021.
- Wang, W., Fang, B., Jing, C., Shen, Y., Shen, Y., Wang, Q., Ouyang, H., Chen, H., and Shen, C. Time is a feature: Exploiting temporal dynamics in diffusion language models. *CoRR*, abs/2508.09138, 2025a.
- Wang, X., Xu, C., Jin, Y., Jin, J., Zhang, H., and Deng, Z. Diffusion llms can do faster-than-ar inference via discrete diffusion forcing. *CoRR*, abs/2508.09192, 2025b.
- Wang, Y., Ma, X., Zhang, G., Ni, Y., Chandra, A., Guo, S., Ren, W., Arulraj, A., He, X., Jiang, Z., Li, T., Ku, M., Wang, K., Zhuang, A., Fan, R., Yue, X., and Chen, W. Mmlu-pro: A more robust and challenging multi-task language understanding benchmark. In *NeurIPS*, 2024.

- Wu, C., Zhang, H., Xue, S., Diao, S., Fu, Y., Liu, Z., Molchanov, P. O., Luo, P., Han, S., and Xie, E. Fast-dllm v2: Efficient block-diffusion LLM. *CoRR*, abs/2509.26328, 2025a.
- Wu, C., Zhang, H., Xue, S., Liu, Z., Diao, S., Zhu, L., Luo, P., Han, S., and Xie, E. Fast-dllm: Training-free acceleration of diffusion LLM by enabling KV cache and parallel decoding. *CoRR*, abs/2505.22618, 2025b.
- Yang, L., Tian, Y., Li, B., Zhang, X., Shen, K., Tong, Y., and Wang, M. Mmada: Multimodal large diffusion language models. *CoRR*, abs/2505.15809, 2025.
- Ye, J., Xie, Z., Zheng, L., Gao, J., Wu, Z., Jiang, X., Li, Z., and Kong, L. Dream 7b: Diffusion large language models. *CoRR*, abs/2508.15487, 2025.
- You, Z., Nie, S., Zhang, X., Hu, J., Zhou, J., Lu, Z., Wen, J., and Li, C. Llada-v: Large language diffusion models with visual instruction tuning. *CoRR*, abs/2505.16933, 2025.
- Zhang, B. and Sennrich, R. Root mean square layer normalization. In *NeurIPS*, pp. 12360–12371, 2019.
- Zhang, D., Song, J., Bi, Z., Yuan, Y., Wang, T., Yeong, J., and Hao, J. Mixture of experts in large language models. *CoRR*, abs/2507.11181, 2025a.
- Zhang, J., Wang, H., Jiang, K., Yang, S., Zheng, K., Xi, H., Wang, Z., Zhu, H., Zhao, M., Stoica, I., Gonzalez, J. E., Zhu, J., and Chen, J. SLA: beyond sparsity in diffusion transformers via fine-tunable sparse-linear attention. *CoRR*, abs/2509.24006, 2025b.
- Zhang, K., Yao, Q., Liu, S., Wang, Y., Lai, B., Ye, J., Song, M., and Tao, D. Consistent paths lead to truth: Self-rewarding reinforcement learning for LLM reasoning. *CoRR*, abs/2506.08745, 2025c.
- Zhao, S., Gupta, D., Zheng, Q., and Grover, A. d1: Scaling reasoning in diffusion large language models via reinforcement learning. *CoRR*, abs/2504.12216, 2025.
- Zhu, F., Wang, R., Nie, S., Zhang, X., Wu, C., Hu, J., Zhou, J., Chen, J., Lin, Y., Wen, J., and Li, C. Llada 1.5: Variance-reduced preference optimization for large language diffusion models. *CoRR*, abs/2505.19223, 2025.

## A Proof of Theorems in Section 3

### A.1 Proof of Theorem 3.1

*Proof.* We adopt two stability assumptions:

1. **Stable Attention:** The total variation of attention weights is bounded:  $\sum_j |\alpha_{ij}^{t+1} - \alpha_{ij}^t| \leq \delta_A$ .
2. **Maximal Drift at  $i$ :** The drift of the  $i$ -th Value vector upper-bounds the weighted drift of the context:  $\sum_j \alpha_{ij}^{t+1} \|\mathbf{v}_j^{t+1} - \mathbf{v}_j^t\| \leq \lambda \|\mathbf{v}_i^{t+1} - \mathbf{v}_i^t\|$ , where  $\lambda \geq 1$  is a relax constant.

We define the drift of the output vector as  $\Delta \mathbf{h}_i = \mathbf{h}_i^{t+1} - \mathbf{h}_i^t$ . Using the triangle inequality, we separate the drift into Value shifts and Attention shifts:

$$\|\Delta \mathbf{h}_i\| = \left\| \sum_j \alpha_{ij}^{t+1} \mathbf{v}_j^{t+1} - \sum_j \alpha_{ij}^t \mathbf{v}_j^t \right\| \leq \underbrace{\sum_j \alpha_{ij}^{t+1} \|\mathbf{v}_j^{t+1} - \mathbf{v}_j^t\|}_{\text{Weighted Value Drift } (\mathcal{D}_v)} + \underbrace{\sum_j |\alpha_{ij}^{t+1} - \alpha_{ij}^t| \|\mathbf{v}_j^t\|}_{\text{Attention Shift } (\mathcal{E}_\alpha)}. \quad (6)$$

For the value drift term  $\mathcal{D}_v$ :

$$\mathcal{D}_v \leq \lambda \|\mathbf{v}_i^{t+1} - \mathbf{v}_i^t\|. \quad (7)$$

Let the norms be bounded such that  $\|\mathbf{v}\| \in [V_{\min}, V_{\max}]$  and  $\|\mathbf{h}\| \in [H_{\min}, H_{\max}]$  (Ruibin et al., 2020; Jacob & Ambuj, 2024). For the attention shift term  $\mathcal{E}_\alpha$ , using the bound  $\delta_A$  and max norm  $V_{\max}$ :

$$\mathcal{E}_\alpha \leq \delta_A V_{\max}. \quad (8)$$

Let  $\Delta_V = V_{\max} - V_{\min}$  be the maximum norm fluctuation.

$$\|\mathbf{v}_i^{t+1} - \mathbf{v}_i^t\| = \sqrt{(\|\mathbf{v}_i^{t+1}\| - \|\mathbf{v}_i^t\|)^2 + 2\|\mathbf{v}_i^{t+1}\|\|\mathbf{v}_i^t\|(1 - \mathcal{S}_{\cos}(\mathbf{v}_i^t, \mathbf{v}_i^{t+1}))} \quad (9)$$

$$\leq \sqrt{\Delta_V^2 + 2V_{\max}^2(1 - \mathcal{S}_{\cos}(\mathbf{v}_i^t, \mathbf{v}_i^{t+1}))} \quad (10)$$

Using the subadditivity of the square root ( $\sqrt{a+b} \leq \sqrt{a} + \sqrt{b}$ ):

$$\|\mathbf{v}_i^{t+1} - \mathbf{v}_i^t\| \leq \Delta_V + V_{\max} \sqrt{2(1 - \mathcal{S}_{\cos}(\mathbf{v}_i^t, \mathbf{v}_i^{t+1}))}. \quad (11)$$

Substituting this back into the bound for  $\|\Delta \mathbf{h}_i\|$ :

$$\|\Delta \mathbf{h}_i\| \leq \lambda V_{\max} \sqrt{2(1 - \mathcal{S}_{\cos}(\mathbf{v}_i^t, \mathbf{v}_i^{t+1}))} + (\lambda \Delta_V + \delta_A V_{\max}). \quad (12)$$

Let  $X = \lambda V_{\max} \sqrt{2(1 - \mathcal{S}_{\cos}(\mathbf{v}_i^t, \mathbf{v}_i^{t+1}))}$  be the signal term and  $Y = \lambda \Delta_V + \delta_A V_{\max}$  be the noise term.

From the geometry of cosine similarity:

$$\mathcal{S}_{\cos}(\mathbf{h}_i^t, \mathbf{h}_i^{t+1}) \geq 1 - \frac{\|\Delta \mathbf{h}_i\|^2}{2H_{\min}^2}. \quad (13)$$

Rearranging for dissimilarity ( $1 - \mathcal{S}$ ):

$$1 - \mathcal{S}_{\cos}(\mathbf{h}_i^t, \mathbf{h}_i^{t+1}) \leq \frac{(X + Y)^2}{2H_{\min}^2} \leq \frac{X^2 + 2XY + Y^2}{2H_{\min}^2}. \quad (14)$$

Substituting  $X^2 = 2\lambda^2 V_{\max}^2 (1 - \mathcal{S}_{\cos}(\mathbf{v}_i^t, \mathbf{v}_i^{t+1}))$ :

$$1 - \mathcal{S}_{\cos}(\mathbf{h}_i^t, \mathbf{h}_i^{t+1}) \leq \underbrace{\frac{\lambda^2 V_{\max}^2}{H_{\min}^2} (1 - \mathcal{S}_{\cos}(\mathbf{v}_i^t, \mathbf{v}_i^{t+1}))}_C + \underbrace{\frac{2XY + Y^2}{2H_{\min}^2}}_\epsilon. \quad (15)$$

This concludes the proof. The output dissimilarity is linearly bounded by the input dissimilarity of token  $i$ , scaled by the squared condition number  $C = (\lambda V_{\max}/H_{\min})^2$ .  $\square$

## A.2 Proof of Theorem 3.2

*Proof.* A standard Transformer MLP consists of RMSNorm operations, several linear projections, activation functions  $\sigma(\cdot)$ . Each of these components is Lipschitz continuous (Henry et al., 2018):

- Linear transformations ( $\mathbf{W}$ ) are Lipschitz with constant equal to their spectral norm  $\|\mathbf{W}\|$ .
- Standard activations (ReLU, GELU, SiLU) are Lipschitz continuous with constant  $L_{\text{act}} \approx 1$ .
- RMSNorm is Lipschitz continuous over a domain of inputs bounded away from zero.

The composition of Lipschitz functions is Lipschitz. Thus, there exists a constant  $L$  such that for any inputs  $\mathbf{h}_1, \mathbf{h}_2$ :

$$\|f_{\text{MLP}}(\mathbf{h}_1) - f_{\text{MLP}}(\mathbf{h}_2)\| \leq L\|\mathbf{h}_1 - \mathbf{h}_2\|. \quad (16)$$

Consider the squared Euclidean distance between the inputs  $\mathbf{h}_1$  and  $\mathbf{h}_2$ :

$$\|\mathbf{h}_1 - \mathbf{h}_2\|^2 = \|\mathbf{h}_1\|^2 + \|\mathbf{h}_2\|^2 - 2\|\mathbf{h}_1\|\|\mathbf{h}_2\|\mathcal{S}_{\cos}(\mathbf{h}_1, \mathbf{h}_2). \quad (17)$$

Assuming the hidden states within the Transformer operate within a bounded norm range (Ruibin et al., 2020; Jacob & Ambuj, 2024), let  $H_{\min} \leq \|\mathbf{h}\| \leq H_{\max}$ . The Euclidean distance can be exactly related to cosine similarity via:

$$\|\mathbf{h}_1 - \mathbf{h}_2\|^2 = (\|\mathbf{h}_1\| - \|\mathbf{h}_2\|)^2 + 2\|\mathbf{h}_1\|\|\mathbf{h}_2\|(1 - \mathcal{S}_{\cos}(\mathbf{h}_1, \mathbf{h}_2)). \quad (18)$$

Bounding the magnitude difference by  $\Delta = H_{\max} - H_{\min}$  and the product by  $H_{\max}^2$ , we obtain a strict upper bound:

$$\|\mathbf{h}_1 - \mathbf{h}_2\|^2 \leq \Delta^2 + 2H_{\max}^2(1 - \mathcal{S}_{\cos}(\mathbf{h}_1, \mathbf{h}_2)). \quad (19)$$

Using the subadditivity of the square root ( $\sqrt{a+b} \leq \sqrt{a} + \sqrt{b}$ ):

$$\|\mathbf{h}_1 - \mathbf{h}_2\| \leq \Delta + \sqrt{2}H_{\max} \cdot \sqrt{1 - \mathcal{S}_{\cos}(\mathbf{h}_1, \mathbf{h}_2)}. \quad (20)$$

Substituting Equation (20) into the Lipschitz condition (Equation (16)), we set  $C = L \cdot \sqrt{2}H_{\max}$  and  $\epsilon = L \cdot \Delta$ :

$$\|f_{\text{MLP}}(\mathbf{h}_1) - f_{\text{MLP}}(\mathbf{h}_2)\| \leq \epsilon + C \cdot \sqrt{1 - \mathcal{S}_{\cos}(\mathbf{h}_1, \mathbf{h}_2)}. \quad (21)$$

□

## A.3 Proof of Theorem 3.4

### A.3.1 PROPOSITIONS AND LEMMA FOR THE PROOF

**Proposition A.1. (Generalized Bernoulli Theorem)** For  $x \geq 0$ ,  $1/\sqrt{1+x} \geq 1 - x/2$ .

*Proof.* The proof is straightforward by Taylor expansion of  $1/\sqrt{1+x}$  at  $x = 0$ :

$$\frac{1}{\sqrt{1+x}} = 1 - \frac{1}{2}x + \underbrace{\frac{3}{8}x^2 - \frac{5}{16}x^3 + \dots}_{\geq 0} \geq 1 - \frac{1}{2}x. \quad (22)$$

□

**Proposition A.2.** For any  $\mathbf{h} \in \mathbb{R}^d$  and the  $r$ -rank approximation  $\mathbf{U}_r \Lambda_r \mathbf{V}_r^\top$  of  $\mathbf{W}$ , the following inequality holds:

$$\frac{\|\mathbf{W}\mathbf{h} - \mathbf{U}_r \Lambda_r \mathbf{V}_r^\top \mathbf{h}\|}{\|\Lambda_r \mathbf{V}_r^\top \mathbf{h}\|} \leq \frac{\lambda_{r+1}}{\lambda_r} \frac{\|\mathbf{h}\|}{\|\mathbf{V}_r^\top \mathbf{h}\|}. \quad (23)$$

*Proof.* For the numerator, we have

$$\|\mathbf{W}\mathbf{h} - \mathbf{U}_r\Lambda_r\mathbf{V}_r^\top\mathbf{h}\| \leq \|\mathbf{W} - \mathbf{U}_r\Lambda_r\mathbf{V}_r^\top\| \|\mathbf{h}\| = \lambda_{r+1}\|\mathbf{h}\|. \quad (24)$$

For the denominator, we have

$$\|\Lambda_r\mathbf{V}_r^\top\mathbf{h}\| \geq \lambda_r\|\mathbf{V}_r^\top\mathbf{h}\|. \quad (25)$$

Combining these two inequalities,

$$\frac{\|\mathbf{W}\mathbf{h} - \mathbf{U}_r\Lambda_r\mathbf{V}_r^\top\mathbf{h}\|}{\|\Lambda_r\mathbf{V}_r^\top\mathbf{h}\|} \leq \frac{\lambda_{r+1}}{\lambda_r} \frac{\|\mathbf{h}\|}{\|\mathbf{V}_r^\top\mathbf{h}\|}. \quad (26)$$

□

**Lemma A.3.** Let  $\mathbf{W}_r = \Lambda_r\mathbf{V}_r^\top$ ,  $\mathbf{o} = \mathbf{W}\mathbf{h}$ , and  $\mathbf{o}' = \mathbf{W}_r\mathbf{h}$ . Then the divergence of the cosine similarity between the full and low-dimensional projections can be bounded as follows:

$$|\mathcal{S}_{\cos}(\mathbf{o}_1, \mathbf{o}_2) - \mathcal{S}_{\cos}(\mathbf{o}'_1, \mathbf{o}'_2)| \leq \frac{1}{2} \left( \frac{\lambda_{r+1}}{\lambda_r} \right)^2 \left( \frac{\|\mathbf{h}_1\|}{\|\mathbf{V}_r^\top\mathbf{h}_1\|} + \frac{\|\mathbf{h}_2\|}{\|\mathbf{V}_r^\top\mathbf{h}_2\|} \right)^2. \quad (27)$$

*Proof.* We start with residuals of the low-rank approximation  $\mathbf{R} := \mathbf{W} - \mathbf{U}_r\Lambda_r\mathbf{V}_r^\top$ . By the definition of spectral norm, we have  $\|\mathbf{R}\| = \lambda_{r+1}$ . And  $\mathbf{R}$  and  $\mathbf{U}_r$  are orthogonal (mathematically,  $\mathbf{R}^\top\mathbf{U}_r = 0$ ) because the residual  $\mathbf{R}$  lives in the span of the discarded singular vectors  $\text{span}(\mathbf{U}) \setminus \text{span}(\mathbf{U}_r)$ .

Denote  $\mathbf{A} := \mathbf{W} - \mathbf{R} = \mathbf{U}_r\Lambda_r\mathbf{V}_r^\top = \mathbf{U}_r\mathbf{W}_r$ . As the orthogonal matrix  $\mathbf{U}_r$  preserves the inner product and norm,  $\mathcal{S}_{\cos}(\mathbf{o}'_1, \mathbf{o}'_2) = \mathcal{S}_{\cos}(\mathbf{W}_r\mathbf{h}_1, \mathbf{W}_r\mathbf{h}_2) = \mathcal{S}_{\cos}(\mathbf{A}\mathbf{h}_1, \mathbf{A}\mathbf{h}_2)$ , we use  $\mathbf{A}$  rather than  $\mathbf{W}_r$  in the following proof for clarity.

Since  $\mathbf{R}$  and  $\mathbf{A}$  are orthogonal:  $\mathbf{R}^\top\mathbf{A} = \mathbf{R}^\top\mathbf{U}_r\Lambda_r\mathbf{V}_r^\top = 0 \cdot \Lambda_r\mathbf{V}_r^\top = 0$ , the norm of  $\mathbf{W}\mathbf{h}_1$  can be decomposed as follows:

$$\|\mathbf{o}_1\|^2 = \|\mathbf{W}\mathbf{h}_1\|^2 = \|\mathbf{A}\mathbf{h}_1 + \mathbf{R}\mathbf{h}_1\|^2 = \|\mathbf{A}\mathbf{h}_1\|^2 + \|\mathbf{R}\mathbf{h}_1\|^2 + \underbrace{2\langle \mathbf{A}\mathbf{h}_1, \mathbf{R}\mathbf{h}_1 \rangle}_{=0 \text{ as } \mathbf{R}^\top\mathbf{A}=0}. \quad (28)$$

Similarly, the inner products between the adjacent features can be decomposed as follows:

$$\begin{aligned} \langle \mathbf{o}_1, \mathbf{o}_2 \rangle &= \langle \mathbf{W}\mathbf{h}_1, \mathbf{W}\mathbf{h}_2 \rangle = \langle (\mathbf{A} + \mathbf{R})\mathbf{h}_1, (\mathbf{A} + \mathbf{R})\mathbf{h}_2 \rangle \\ &= \langle \mathbf{A}\mathbf{h}_1, \mathbf{A}\mathbf{h}_2 \rangle + \langle \mathbf{R}\mathbf{h}_1, \mathbf{R}\mathbf{h}_2 \rangle + \underbrace{\langle \mathbf{A}\mathbf{h}_1, \mathbf{R}\mathbf{h}_2 \rangle + \langle \mathbf{R}\mathbf{h}_1, \mathbf{A}\mathbf{h}_2 \rangle}_{=0 \text{ as } \mathbf{R}^\top\mathbf{A}=0}. \end{aligned} \quad (29)$$

Denote  $a_1 := \|\mathbf{A}\mathbf{h}_1\|$ ,  $h_1 := \|\mathbf{W}\mathbf{h}_1\|$ , and  $r_1 := \|\mathbf{R}\mathbf{h}_1\|$  for brevity,

$$\begin{aligned} \mathcal{S}_{\cos}(\mathbf{o}_1, \mathbf{o}_2) - \mathcal{S}_{\cos}(\mathbf{o}'_1, \mathbf{o}'_2) &= \frac{\langle \mathbf{W}\mathbf{h}_1, \mathbf{W}\mathbf{h}_2 \rangle}{\|\mathbf{W}\mathbf{h}_1\| \|\mathbf{W}\mathbf{h}_2\|} - \frac{\langle \mathbf{A}\mathbf{h}_1, \mathbf{A}\mathbf{h}_2 \rangle}{\|\mathbf{A}\mathbf{h}_1\| \|\mathbf{A}\mathbf{h}_2\|} \\ &= \frac{\langle \mathbf{A}\mathbf{h}_1, \mathbf{A}\mathbf{h}_2 \rangle + \langle \mathbf{R}\mathbf{h}_1, \mathbf{R}\mathbf{h}_2 \rangle}{\|\mathbf{A}\mathbf{h}_1\| \|\mathbf{A}\mathbf{h}_2\|} \cdot \frac{\|\mathbf{A}\mathbf{h}_1\| \|\mathbf{A}\mathbf{h}_2\|}{\|\mathbf{W}\mathbf{h}_1\| \|\mathbf{W}\mathbf{h}_2\|} - \frac{\langle \mathbf{A}\mathbf{h}_1, \mathbf{A}\mathbf{h}_2 \rangle}{\|\mathbf{A}\mathbf{h}_1\| \|\mathbf{A}\mathbf{h}_2\|} \quad [\text{by Equation (29)}] \\ &= \underbrace{\frac{\langle \mathbf{A}\mathbf{h}_1, \mathbf{A}\mathbf{h}_2 \rangle}{\|\mathbf{A}\mathbf{h}_1\| \|\mathbf{A}\mathbf{h}_2\|}}_{=\mathcal{S}_{\cos}(\mathbf{o}'_1, \mathbf{o}'_2) \in [-1, 1]} \underbrace{\left( \frac{\|\mathbf{A}\mathbf{h}_1\| \|\mathbf{A}\mathbf{h}_2\|}{\|\mathbf{W}\mathbf{h}_1\| \|\mathbf{W}\mathbf{h}_2\|} - 1 \right)}_{=\frac{a_1 a_2}{h_1 h_2} - 1} + \frac{\langle \mathbf{R}\mathbf{h}_1, \mathbf{R}\mathbf{h}_2 \rangle}{\|\mathbf{W}\mathbf{h}_1\| \|\mathbf{W}\mathbf{h}_2\|}. \end{aligned} \quad (30)$$

For the absolute value of the first term, we have

$$|\mathcal{S}_{\cos}(\mathbf{o}'_1, \mathbf{o}'_2)| \left| \frac{a_1 a_2}{h_1 h_2} - 1 \right| \leq \frac{1}{2} \left( \frac{r_1}{a_1} \right)^2 + \frac{1}{2} \left( \frac{r_2}{a_2} \right)^2. \quad (31)$$

Here the inequality is derivated from Proposition A.1 by applying  $x = \left( \frac{r_1}{a_1} \right)^2$ :

$$\frac{a_1}{h_1} = \frac{1}{\sqrt{1 + \left( \frac{r_1}{a_1} \right)^2}} \geq 1 - \frac{1}{2} \left( \frac{r_1}{a_1} \right)^2 \text{ and similarly, } \frac{a_2}{h_2} \geq 1 - \frac{1}{2} \left( \frac{r_2}{a_2} \right)^2, \quad (32)$$

$$\frac{a_1 a_2}{h_1 h_2} \geq 1 - \frac{1}{2} \left( \frac{r_1}{a_1} \right)^2 - \frac{1}{2} \left( \frac{r_2}{a_2} \right)^2 + \frac{1}{4} \left( \frac{r_1 r_2}{a_1 a_2} \right)^2 \geq 1 - \frac{1}{2} \left( \frac{r_1}{a_1} \right)^2 - \frac{1}{2} \left( \frac{r_2}{a_2} \right)^2. \quad (33)$$

So,

$$\left| \frac{a_1 a_2}{h_1 h_2} - 1 \right| = 1 - \frac{a_1 a_2}{h_1 h_2} \leq \frac{1}{2} \left( \frac{r_1}{a_1} \right)^2 + \frac{1}{2} \left( \frac{r_2}{a_2} \right)^2. \quad (34)$$

For the absolute value of the second term, we can use the Cauchy-Schwarz inequality:

$$\left| \frac{\langle \mathbf{R} \mathbf{h}_1, \mathbf{R} \mathbf{h}_2 \rangle}{\|\mathbf{W} \mathbf{h}_1\| \|\mathbf{W} \mathbf{h}_2\|} \right| \leq \frac{\|\mathbf{R} \mathbf{h}_1\| \|\mathbf{R} \mathbf{h}_2\|}{\|\mathbf{W} \mathbf{h}_1\| \|\mathbf{W} \mathbf{h}_2\|} = \frac{r_1 r_2}{h_1 h_2} \leq \frac{r_1 r_2}{a_1 a_2} \quad (35)$$

Combining Equations (31), (35) and (30), we have

$$|\mathcal{S}_{\cos}(\mathbf{o}_1, \mathbf{o}_2) - \mathcal{S}_{\cos}(\mathbf{o}'_1, \mathbf{o}'_2)| \leq \frac{1}{2} \left( \frac{r_1}{a_1} \right)^2 + \frac{1}{2} \left( \frac{r_2}{a_2} \right)^2 + \frac{r_1 r_2}{a_1 a_2} = \frac{1}{2} \left( \frac{r_1}{a_1} + \frac{r_2}{a_2} \right)^2. \quad (36)$$

By Proposition A.2,

$$\frac{r_1}{a_1} \leq \frac{\lambda_{r+1}}{\lambda_r} \frac{\|\mathbf{h}_1\|}{\|\mathbf{V}_r^\top \mathbf{h}_1\|}, \text{ and } \frac{r_2}{a_2} \leq \frac{\lambda_{r+1}}{\lambda_r} \frac{\|\mathbf{h}_2\|}{\|\mathbf{V}_r^\top \mathbf{h}_2\|}. \quad (37)$$

Substituting Equation (37) into Equation (36), the final bound is

$$|\mathcal{S}_{\cos}(\mathbf{o}_1, \mathbf{o}_2) - \mathcal{S}_{\cos}(\mathbf{o}'_1, \mathbf{o}'_2)| \leq \frac{1}{2} \left( \frac{\lambda_{r+1}}{\lambda_r} \right)^2 \left( \frac{\|\mathbf{h}_1\|}{\|\mathbf{V}_r^\top \mathbf{h}_1\|} + \frac{\|\mathbf{h}_2\|}{\|\mathbf{V}_r^\top \mathbf{h}_2\|} \right)^2. \quad (38)$$

□

### A.3.2 PROOF OF THEOREM 3.4

*Proof.* With Lemma A.3, it suffices to show that for any  $\mathbf{h}_1, \mathbf{h}_2 \in \text{span}(\mathbf{V}_r)$ , we have  $\|\mathbf{h}_1\| = \|\mathbf{V}_r^\top \mathbf{h}_1\|$  and  $\|\mathbf{h}_2\| = \|\mathbf{V}_r^\top \mathbf{h}_2\|$ , which establishes the conclusion of Theorem 3.4. This is straightforward because  $\mathbf{V}_r$  is an orthonormal basis of  $\text{span}(\mathbf{V}_r)$ , so for any  $\mathbf{h} \in \text{span}(\mathbf{V}_r)$ , we have  $\|\mathbf{h}\| = \|\mathbf{V}_r^\top \mathbf{h}\|$ . □

**Remark A.4. (Connection with MaxLogits)** It is widely observed that the spectral norm of the attention projection matrix,  $\|\mathbf{W}\|_2 = \lambda_1$ , tends to become large during the training of large language models. While this behavior can lead to numerical instability (Steven et al., 2022), it also incidentally benefits the error bound of our proposed singular proxy identifier.

## B Failure Analysis

As discussed in Section 3.2, using the attention output as the update identifier has the worst accuracy among all configurations in Table 1. Intuitively, the attention output should be eligible. We then dive deeper to investigate its failure mechanism. The core issue lies in that the anisotropy of the attention outputs have been masked, reducing the sensitivity of the similarity metric to subtle semantic drifts.

Specifically, the anisotropy masks individual token characteristics, thereby reducing the sensitivity of the similarity metric to subtle semantic drifts (Ethayarajh, 2019; Gao et al., 2019; Timkey & van Schijndel, 2021; Zhang et al., 2025c; Fang et al., 2025). It is widely observed that hidden states in Transformers, specifically the Value states  $\mathbf{v}_i$  in our context, can be decomposed into a common vector  $\mathbf{c}$  and an independent semantic signal  $\mathbf{s}_i$ :

$$\mathbf{v}_i = \mathbf{c} + \mathbf{s}_i, \quad \text{where } \mathbb{E}[\mathbf{s}_i] = \mathbf{0} \text{ and } \|\mathbf{c}\| > 0. \quad (39)$$

While this phenomenon is present, the common vector  $\mathbf{c}$  remains relatively non-dominant ( $\|\mathbf{s}_i\| > \|\mathbf{c}\|$ ), allowing the states to maintain sufficient orthogonality ( $\mathbb{E}_{i \neq j}[\mathcal{S}_{\cos}(\mathbf{v}_i, \mathbf{v}_j)] \approx 0$ ). This preservation of semantic variance is visualized in

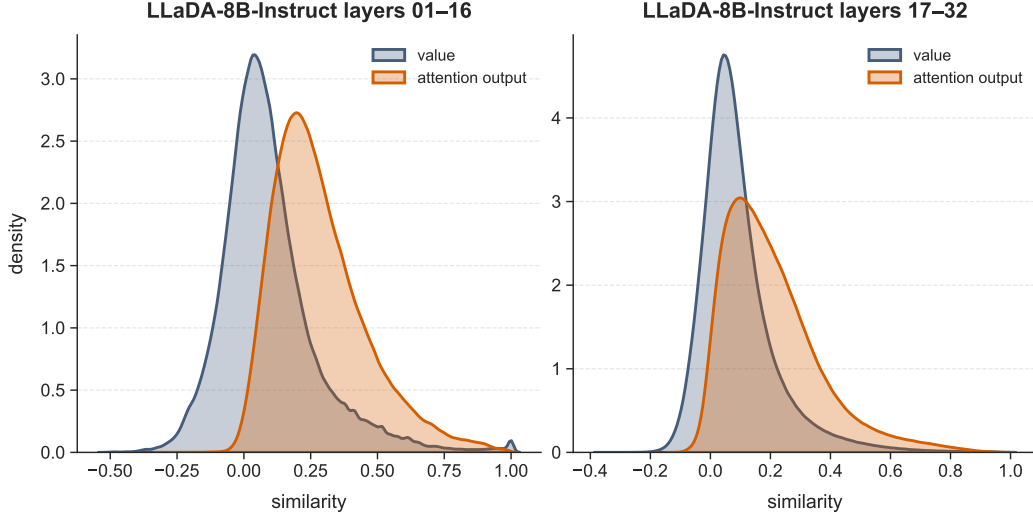


Figure 5. **Empirical Evidence of Anisotropy Problem in Attention Outputs.** The density plots compare the cosine similarity distributions of value states (blue) versus attention outputs (orange). While value states exhibit an isotropic distribution centered near zero (orthogonality), the attention outputs show a significant positive shift in mean similarity.

Table 6. Hyperparameters for fitted piecewise Gaussian function.

| MODEL                | $l_p$ | $\rho_p$ | $\rho_1$ | $\rho_L$ |
|----------------------|-------|----------|----------|----------|
| LLADA-8B-INSTRUCT    | 24    | 25%      | 3%       | 13%      |
| LLADA-1.5            | 25    | 25%      | 3%       | 13%      |
| DREAM-V0-INSTRUCT-7B | 14    | 30%      | 5%       | 25%      |

Figure 5. However, this representation collapses when states are aggregated into the attention output  $\mathbf{h}_i = \sum_j \alpha_{ij} \mathbf{v}_j$ . Given the stochastic constraint  $\sum_j \alpha_{ij} = 1$ , the transformation is expressed as:

$$\mathbf{h}_i = \sum_j \alpha_{ij} (\mathbf{c} + \mathbf{s}_j) = \mathbf{c} + \sum_j \alpha_{ij} \mathbf{s}_j. \quad (40)$$

Here, the attention output  $\mathbf{h}_i$  becomes heavily dominated by the common direction  $\mathbf{c}$ . Because the semantic signals  $\mathbf{s}_j$  are independent and zero-mean, their weighted sum undergoes a form of “signal cancellation,” leading to  $\|\mathbf{c}\| \gg \|\sum_j \alpha_{ij} \mathbf{s}_j\|$ . Consequently, the similarity distribution  $\mathcal{S}_{\cos}(\mathbf{h}_i, \mathbf{h}_j)$  shifts toward a narrow cone where cosine similarities approach 1. This anisotropy renders the identifier insensitive to the subtle semantic drifts required for accurate update tracking.

## C Details on Adaptive Budget Allocation

In Section 3.4, we characterized the layer-wise heterogeneity of hidden state dynamics for LLaDA-8B (Nie et al., 2025). In this section, we extend our analysis to a broader range of models. Specifically, the distribution profiles for LLaDA-1.5B (Zhu et al., 2025) and Dream-7B (Ye et al., 2025) are illustrated in Figure 6.

While these models exhibit a general trend consistent with LLaDA-8B, the parameters for the fitted piecewise Gaussian distributions vary across architectures. We provide the specific hyperparameters employed for these models in Table 6. It is important to emphasize that these hyperparameters remain fixed across all evaluated benchmarks, demonstrating the robust generalization capabilities of our proposed method.

## D Experimental Details

In our experiments, we follow the decoding hyperparameters established by Liu et al. (2025b). For completeness, these detailed configurations are provided in Table 7.

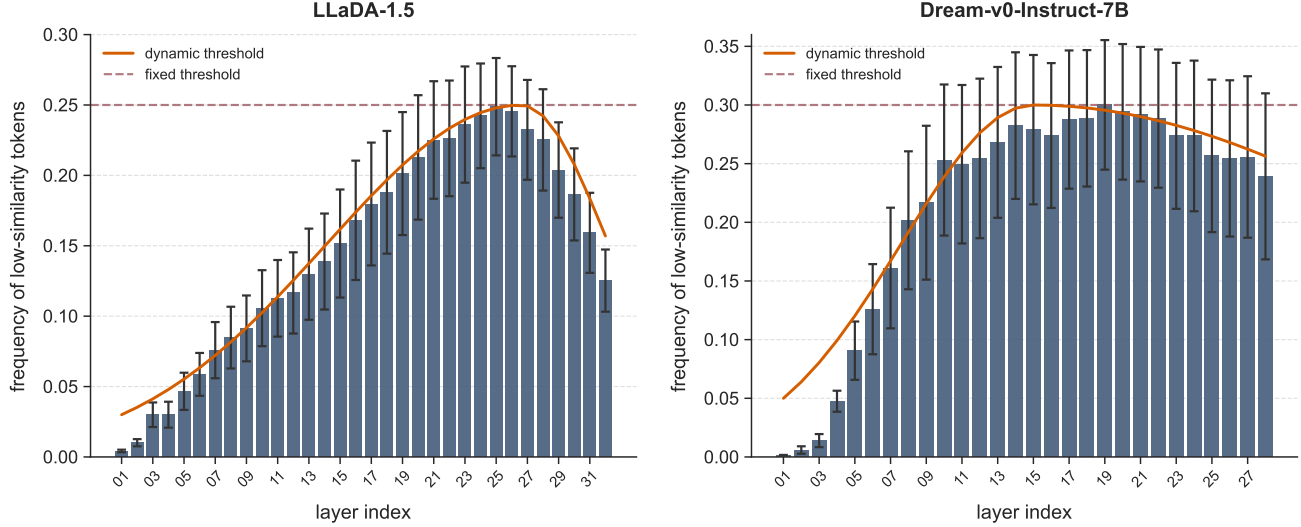


Figure 6. Distribution of drift for LLaDA-1.5 (Zhu et al., 2025) and Dream-v0-Instruct-7B (Ye et al., 2025).

Table 7. Experiment settings across benchmarks.

| TASK      | LLADA-8B-INSTRUCT & LLADA-1.5 |             |              | DREAM-v0-INSTRUCT-7B |             |              |
|-----------|-------------------------------|-------------|--------------|----------------------|-------------|--------------|
|           | N-SHOT                        | GEN. LENGTH | BLOCK LENGTH | N-SHOT               | GEN. LENGTH | BLOCK LENGTH |
| GSM8K     | 4                             | 256         | 8            | 8                    | 256         | 256          |
| GPQA      | 5                             | 128         | 64           | 5                    | 256         | 256          |
| MATH500   | 4                             | 256         | 32           | 4                    | 256         | 256          |
| BBH       | 3                             | 256         | 256          | 3                    | 256         | 256          |
| MMLU-PRO  | 5                             | 256         | 256          | 5                    | 256         | 256          |
| MBPP      | 3                             | 512         | 32           | 3                    | 256         | 256          |
| HUMANEVAL | 0                             | 512         | 32           | 0                    | 256         | 256          |

For the Fast-dLLM baseline (Wu et al., 2025b), which requires the block size to be smaller than the generation length to achieve effective acceleration, we set the block size to 32 uniformly across all models and benchmarks. Other method-specific hyperparameters, such as the refresh interval in dLLM-Cache (Liu et al., 2025b), are kept consistent with their original configurations.

Regarding batch sizes, we use a batch size of 16 for all LLaDA-8B tasks. For the Dream-7B model, the batch size is set to 4. This specific reduction was necessitated by Out-of-Memory (OOM) issues encountered with Fast-dLLM during long-prompt benchmarks (e.g. MMLU-pro). While other evaluated methods could support larger batch sizes for higher throughput, we maintain a consistent batch size of 4 for Dream-7B to ensure a fair comparison across all baselines.

Regarding the selection of the proxy rank  $r$ , we set  $r = 128$  for the LLaDA models (Nie et al., 2025; Zhu et al., 2025), where the hidden dimension is  $d = 4096$ . This choice is consistent with our ablation studies detailed in Section 4.4. For the Dream model (Ye et al., 2025), which utilizes Grouped-Query Attention (GQA) (Ainslie et al., 2023), the Value projection dimension is significantly smaller ( $d = 512$ ). Consequently, we adopt a more aggressive proxy rank of  $r = 32$  to maintain a comparable acceleration.

## E Additional Experiment Results

We further evaluate our approach on LLaDA-1.5B (Zhu et al., 2025), with results detailed in Table 8. The performance trends closely mirror those observed for the larger LLaDA-8B and Dream-7B models (Table 2), underscoring the architectural robustness of our method. Notably, SPA-Cache consistently establishes a new state-of-the-art across all benchmarks, delivering substantial throughput gains while preserving the original model’s generation quality with negligible degradation.

We also compare with dKV-Cache (Ma et al., 2025), d2Cache (Jiang et al., 2025), and ElasticCache (Nguyen-Tri et al., 2025)

Table 8. Comparison of different caching methods on LLaDA-1.5 (Zhu et al., 2025).

| TASK                  | METHOD       | TPS $\uparrow$          | TTFT (ms) $\downarrow$ | ACCURACY $\uparrow$ | MEM. (GB) $\downarrow$ |
|-----------------------|--------------|-------------------------|------------------------|---------------------|------------------------|
| MATHEMATICS & SCIENCE |              |                         |                        |                     |                        |
| GSM8K                 | BASILINE     | 28.9 ( $\times 1.0$ )   | 28.04                  | 81.96 ( $\pm 1.1$ ) | 62.3                   |
|                       | + DLLM-CACHE | 68.5 ( $\times 2.4$ )   | 28.03                  | 81.43 ( $\pm 1.1$ ) | 63.8                   |
|                       | + FAST-DLLM  | 58.0 ( $\times 2.0$ )   | 28.09                  | 82.18 ( $\pm 1.1$ ) | 64.1                   |
|                       | + OURS       | 189.9 ( $\times 6.6$ )  | 31.13                  | 81.50 ( $\pm 1.1$ ) | 57.5                   |
| GPQA                  | BASILINE     | 21.0 ( $\times 1.0$ )   | 44.90                  | 27.46 ( $\pm 2.1$ ) | 102.4                  |
|                       | + DLLM-CACHE | 29.2 ( $\times 1.4$ )   | 44.72                  | 27.90 ( $\pm 2.1$ ) | 77.0                   |
|                       | + FAST-DLLM  | 39.4 ( $\times 1.9$ )   | 44.26                  | 27.01 ( $\pm 2.1$ ) | 91.4                   |
|                       | + OURS       | 49.2 ( $\times 2.3$ )   | 47.68                  | 27.90 ( $\pm 2.1$ ) | 96.6                   |
| MATH500               | BASILINE     | 35.5 ( $\times 1.0$ )   | 23.23                  | 34.40 ( $\pm 0.6$ ) | 63.4                   |
|                       | + DLLM-CACHE | 74.2 ( $\times 2.1$ )   | 23.28                  | 33.92 ( $\pm 0.6$ ) | 57.9                   |
|                       | + FAST-DLLM  | 83.9 ( $\times 2.4$ )   | 23.19                  | 33.54 ( $\pm 0.6$ ) | 66.5                   |
|                       | + OURS       | 201.8 ( $\times 5.7$ )  | 23.13                  | 34.56 ( $\pm 0.6$ ) | 51.6                   |
| GENERAL QA            |              |                         |                        |                     |                        |
| BBH                   | BASILINE     | 22.3 ( $\times 1.0$ )   | 16.22                  | 56.61 ( $\pm 0.6$ ) | 37.0                   |
|                       | + DLLM-CACHE | 36.0 ( $\times 1.6$ )   | 16.35                  | 57.00 ( $\pm 0.6$ ) | 45.0                   |
|                       | + FAST-DLLM  | 103.9 ( $\times 4.7$ )  | 16.22                  | 54.66 ( $\pm 0.6$ ) | 36.3                   |
|                       | + OURS       | 131.5 ( $\times 5.9$ )  | 16.27                  | 57.43 ( $\pm 0.6$ ) | 34.3                   |
| MMLU-PRO              | BASILINE     | 21.4 ( $\times 1.0$ )   | 43.99                  | 38.51 ( $\pm 0.4$ ) | 95.3                   |
|                       | + DLLM-CACHE | 51.7 ( $\times 2.4$ )   | 42.12                  | 38.30 ( $\pm 0.4$ ) | 87.5                   |
|                       | + FAST-DLLM  | 84.3 ( $\times 4.0$ )   | 43.41                  | 38.31 ( $\pm 0.4$ ) | 79.3                   |
|                       | + OURS       | 233.2 ( $\times 10.9$ ) | 43.90                  | 38.36 ( $\pm 0.4$ ) | 79.1                   |
| CODE                  |              |                         |                        |                     |                        |
| MBPP                  | BASILINE     | 6.7 ( $\times 1.0$ )    | 28.61                  | 40.80               | 70.7                   |
|                       | + DLLM-CACHE | 9.0 ( $\times 1.3$ )    | 28.22                  | 40.80               | 74.2                   |
|                       | + FAST-DLLM  | 13.3 ( $\times 2.0$ )   | 28.25                  | 37.20               | 74.1                   |
|                       | + OURS       | 69.4 ( $\times 10.4$ )  | 29.44                  | 40.00               | 69.5                   |
| HUMAN-EVAL            | BASILINE     | 46.8 ( $\times 1.0$ )   | 14.30                  | 45.83               | 51.2                   |
|                       | + DLLM-CACHE | 43.1 ( $\times 0.9$ )   | 14.28                  | 38.41               | 55.7                   |
|                       | + FAST-DLLM  | 92.8 ( $\times 2.0$ )   | 14.23                  | 41.71               | 50.3                   |
|                       | + OURS       | 143.7 ( $\times 3.1$ )  | 14.23                  | 46.88               | 48.9                   |

in Table 9. Our method outperforms these baselines in throughput (TPS) while maintaining accuracy. The performance gap stems from architectural compatibility: existing methods often rely on explicit attention weight computation, which is incompatible with optimized kernels like FlashAttention. In contrast, our method detects shifts using a singular proxy, which maintains compatibility with FlashAttention and achieves higher acceleration at the same or higher accuracy.

## F Additional Observations

Figure 7 demonstrates that the Value Proxy accurately forecasts FFN output drifts. The near-identical behavior of our proposed singular proxy to the Value Proxy empirically validates Theorem 3.4, confirming that a low-rank approximation effectively captures value-space drift.

Table 9. Comparison against dKV-Cache (Ma et al., 2025), Elastic-Cache (Nguyen-Tri et al., 2025), and d2Cache (Jiang et al., 2025) on LLaDA-8B-Instruct (Nie et al., 2025) and Dream-v0-Instruct-7B (Ye et al., 2025).

| TASK  | METHOD        | LLADA-8B-INSTRUCT                      |                        |                      | DREAM-V0-INSTRUCT-7B                   |                        |                      |
|-------|---------------|----------------------------------------|------------------------|----------------------|----------------------------------------|------------------------|----------------------|
|       |               | TPS $\uparrow$                         | TTFT (ms) $\downarrow$ | ACCURACY $\uparrow$  | TPS $\uparrow$                         | TTFT (ms) $\downarrow$ | ACCURACY $\uparrow$  |
| GSM8K | VANILLA       | 29.67 (1.0 $\times$ )                  | 28.0                   | 78.62 ( $\pm 1.13$ ) | 17.86 (1.0 $\times$ )                  | 23.6                   | 75.21 ( $\pm 1.19$ ) |
|       | DKV-CACHE     | 56.97 (1.9 $\times$ )                  | 28.9                   | 78.01 ( $\pm 1.14$ ) | 22.17 (1.2 $\times$ )                  | 24.4                   | 74.82 ( $\pm 1.20$ ) |
|       | ELASTIC-CACHE | 48.00 (1.6 $\times$ )                  | 27.4                   | 77.41 ( $\pm 1.15$ ) | 35.77 (2.0 $\times$ )                  | 23.9                   | 74.86 ( $\pm 1.22$ ) |
|       | D2CACHE       | 26.85 (0.9 $\times$ )                  | 28.7                   | 76.42 ( $\pm 1.17$ ) | 28.92 (1.6 $\times$ )                  | 24.9                   | 78.70 ( $\pm 1.13$ ) |
|       | OURS          | <b>190.73 (6.4<math>\times</math>)</b> | 28.7                   | 78.24 ( $\pm 1.14$ ) | <b>45.97 (2.6<math>\times</math>)</b>  | 24.5                   | 77.56 ( $\pm 1.15$ ) |
| MBPP  | VANILLA       | 5.75 (1.0 $\times$ )                   | 28.7                   | 39.20                | 36.51 (1.0 $\times$ )                  | 22.2                   | 57.40                |
|       | DKV-CACHE     | 7.83 (1.4 $\times$ )                   | 28.1                   | 39.20                | 39.56 (1.1 $\times$ )                  | 22.2                   | 56.40                |
|       | ELASTIC-CACHE | 16.25 (2.8 $\times$ )                  | 29.2                   | 34.80                | 85.60 (2.3 $\times$ )                  | 22.1                   | 52.80                |
|       | D2CACHE       | 25.08 (4.4 $\times$ )                  | 28.2                   | 39.60                | 41.87 (1.1 $\times$ )                  | 21.6                   | 57.60                |
|       | OURS          | <b>46.12 (8.0<math>\times</math>)</b>  | 28.7                   | 39.00                | <b>114.85 (3.1<math>\times</math>)</b> | 22.6                   | 57.60                |

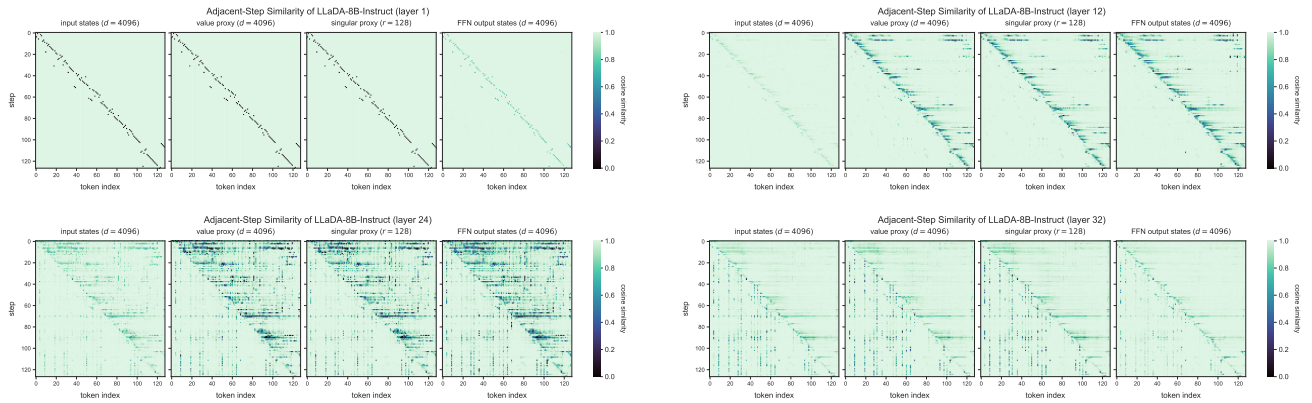


Figure 7. **Extended analysis of adjacent-step state similarities.** We visualize the similarities across four features (input, value, singular proxy, and output) for representative layers (1, 12, 24, and 32) of LLaDA-8B-Instruct as in Figure 1. The value proxy uncovers drift in the final FFN output more effectively than input states. Furthermore, the singular proxy’s near-identical behavior to the value proxy suggests that a low-rank approximation effectively captures value-space drift.